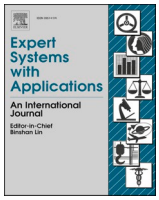
Expert Systems With Applications 255 (2024) 124550



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





Ensemble empirical mode decomposition based deep learning models for forecasting river flow time series

Reetun Maiti [a,d], Balagopal G. Menon [b,c,*], Anand Abraham [b]

[a] Research Intern, Centre for Computational and Data Sciences, Indian Institute of Technology, Kharagpur 721302, West Bengal, India
[b] Department of Industrial and Systems Engineering, Indian Institute of Technology, Kharagpur 721302, West Bengal, India
[c] Centre of Excellence in Safety Engineering & Analytics, Indian Institute of Technology, Kharagpur 721302, West Bengal, India
[d] Currently Graduate Student, Edward P. Fitts Department of Industrial and Systems Engineering, North Carolina State University, Raleigh, USA

ARTICLE INFO

Keywords:
Data characteristics analysis
Empirical mode decomposition
Intrinsic mode functions
Deep learning
Performance measures
Diebold-Mariano test
K-fold cross validation
Prophet

ABSTRACT

River flow forecasting is important for flood prediction and effective utilization of water resources. This study proposed a comprehensive methodology that simultaneously enables the creation of multiple datasets and a comparison of multiple learning models for forecasting river flows. First, the data characteristics of the river flow time series were assessed based on its nature and pattern characteristics. Second, data decomposition was made for simplifying the complex data into decomposed modes. Ensemble Empirical Mode Decomposition (EEMD) was used for this purpose that provides intrinsic mode functions (IMFs). In addition, as benchmark, traditional time series decomposition was also applied over the complex data. Third, based on their data characteristics, the IMFs were combined into different components. These components along with the traditional time series decomposition create multiple datasets. Then, the components from the different datasets were forecasted using deep learning models, mainly BPNN (Back-Propagation Neural Network), CNN (Convoluted Neural Network) and LSTM (Long-Short Term Memory). Then, ensemble prediction was used to get the final output. For verification and benchmark purpose, we have also used statistical models like SARIMA over the datasets. A comprehensive evaluation and model comparisons were made using Diebold-Mariano (DM) test, k-fold cross validation and with the results of Prophet. In addition, the temporal convolution network (TCN) and the gated recurrent unit (GRU) were also modelled. Finally, all the DL modelling was performed for two data situations: with missing data deleted and missing data imputed. An empirical case study was done considering the average of daily time series data of Neeleswaram Hydrological Observation (HO) station over the Periyar river of South India for the period 2017–2021. The empirical assessment was made using several tests for all the different datasets and different learning models for further insights. Our results indicate that the deep learning-based models offer better predictions for complex time series in comparison to traditional statistical models. The DL models applied on the original time series data provided robust predictions with good performance such as low RMSE and high correlation values. However, the DL models' performances are not enhanced through integration with time series decomposition and EEMD. Furthermore, the empirical results and the k-fold cross validation-based test indicate that all the DL models show equivalent performances. Interestingly, the DL models show superior performance over Prophet.

1. Introduction

River flow prediction is very important for water resource management, disaster management and sustainable development (Samui et al., 2023; Feng and Niu, 2021). River flow prediction is a difficult problem because it is a highly nonlinear hydrological process (Li et al., 2022). There is significant amount of literature on prediction using hydrological data. Phatarfod (1989) provides an exhaustive review of such techniques. Most of this literature deals with traditional time series forecasting techniques and its extensions. However, these traditional statistical models do not have the capability to capture nonlinear patterns in daily river flow time series (Rahmani and Fattahi, 2022). Rather the neural network models capture nonlinear dynamic patterns to a far better extent.

* Corresponding author.
E-mail addresses: reetun13@gmail.com (R. Maiti), bgmenon@iem.iitkgp.ac.in (B.G. Menon), anandabraham@iem.iitkgp.ac.in (A. Abraham).

https://doi.org/10.1016/j.eswa.2024.124550
Received 28 March 2023; Received in revised form 16 June 2024; Accepted 18 June 2024
Available online 25 June 2024
0957-4174/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Initial studies in hydrological time series modelling made use of time series methods based on Auto Regressive Moving Average (ARMA) techniques following the analytical framework of Box and Jenkins (1970). An early application of these techniques to hydrological stream time series is by Carlson et al. (1970) who used ARMA model for analysis of stream flow data of St. Lawrence, Missouri, Neva, and Niger rivers. Further, they used the constructed model for a one year ahead forecasting of the Missouri river flow. Following this, many extensions to ARMA models for hydrological applications have been used (Hipel et al., 1977; McLeod et al., 1977). Salas and Obeysekera (1982) demonstrate the use of the generalized partial autocorrelation function (GPAF) and the R and S functions (Gray et al., 1978) to identify the order of the ARMA model. The method was applied on the actual data of the Nile river and Saint Lawrence river. A multiplicative seasonal autoregressive integrated moving average method (ARIMA) was used by Modarres and Eslamian (2006) for streamflow time series modelling of Zayandehrud River in Iran. They checked the models ability to make monthly predictions for the next year. Another application was carried out by Valipour et al. (2012) where monthly inflow forecasting into the Dez dam is carried out using ARMA and ARIMA methods.

Over the past two decades, machine learning methods have also begun to be utilized for the prediction of hydrological time series owing to their generally superior performance in prediction problems. Coulibaly and Baldwin (2005) utilized Recurrent Neural Networks (RNN) to forecast hydrological time series with non-stationarity. The RNN outperformed multivariate adaptive regression splines (MARS) models. This was demonstrated using simulations and through case studies on Great Salt Lake, the Saint-Lawrence River flows, and the Nile River flows. Pulido-Calvo and Portela (2007) carried out short term forecasts of daily river flows of a Portuguese watershed. Gopakumar et al. (2007) checked the applicability of ANN for river flow time series of Achenkoil river with seasonal rainfall effects. Other machine learning techniques like Support Vector Machine (SVM) was used for forecasting of river flow (Yu et al., 2006; Patel and Ramachandran, 2015). SVM and other techniques have also been used for analysis of hydrological data like rainfall (Matos et al., 2013) and for constructing non-linear flow forecasting models using variables like rainfall and evaporation rate (Huang and Tian, 2011). Recently, hydrological modelling based on Deep learning techniques are gaining popularity. The most common methods in use are Long Short Term Memory (Alizadeh et al.,2021; Cho and Kim, 2022), and Convolutional Neural Networks (Ghimire et al., 2021). ANN-MLP model was used by Rahmani and Fattahi (2022) for daily flow data of ten hydrometric stations (in UK) collected over 10 years (2009–2019). CNN and LSTM were used by Feng and Niu (2021) to predict river flow in the Hun River Basin, China. CNN and BiLSTM were also used over daily runoff data to test peak flood prediction capability (Wu et al., 2023). In other areas, LSTM model was used for wind power forecasting where it outperformed the statistical models (Aslam et al., 2023).

Different hybrid models have also been used successfully for time series prediction over the past few years. There has been significant literature on the usage of decomposition-ensemble model for time series prediction. Decomposition-forecasting has now become a very important time series forecasting method. Guo et al. (2023) used CEEMDAN-LSSVR hybrid model for runoff prediction over lower Yellow River. Fang et al. (2023) used FinBERT-VMD-Att-BiGRU hybrid model for crude oil forecasting. A hybrid decomposition ensemble model, VDM-LSTM-GBRT was used by Sun et al. (2022) for predicting daily river flow data for Yangxian station, Han River, China. The proposed model outperformed other models. Another hybrid model employing VMD-LSTM was investigated by Zuo et al. (2020). They stated that the proposed hybrid model is robust and efficient for forecasting nonstationary and nonlinear time series data. Similar results were also suggested by Panahi et al. (2022) where it achieved high accuracy in capturing nonlinear characteristics of river flow of Karoon River, Iran. Zou et al. (2023) used MEMD-BiGAN model for crude oil forecasting with improved risk coverage.

The river flow analysis of Karuvannur river which is a minor tributary of the Periyar is carried out by Anusree and Varghese (2016). Anusree and Varghese (2016) compare Artificial Neural Networks (ANN), Adaptive Neuro-Fuzzy Inference Systems (ANFIS), and Multiple Non Linear Regression (MNLR). ANFIS model was found to yield better daily prediction accuracy than ANN and MLNR. Recent floods on the Periyar river have triggered interest on studying Periyar flows with the intention of analysing flood frequency (Singh and Chinnasamy, 2021). Simulation studies have also been conducted with the objective of developing flood warning systems (Vijayan et al., 2021) and understanding linkages with LULC (Land Use/Land Cover) changes (Sadhwani et al., 2022). However, not enough literature exists on the application of decomposition-ensemble models for river flow forecasting. The intrinsic mode functions (IMFs) generated by decomposition techniques like empirical mode decomposition (EMD) could capture the inner nonlinear factors of the complex daily river flow data.

Under such background, we propose a decomposition-ensemble model for river flow prediction of daily time series data from Neeleswaram Hyrological Observation, Kerala, India for the period 2017–2019. In the decomposition-ensemble model, four main steps were taken, i.e., data decomposition for classifying the original complex data into simpler decomposed components, reconstruction of the components based on data characteristics, individual prediction of each components, and ensemble prediction to get our desired prediction results. The component reconstruction based on data characteristics will help in forming components each with similar characteristics, thus finding the inner factors and reducing computational costs. Many literature provides evidence of model performance improvement and powerful insights using data reconstruction. For example, Tang et al. (2013) predicted nuclear energy consumption using data- characteristics driven ensemble-decomposition model. Yu et al. (2015) used data characteristics driven reconstruction over intrinsic mode functions (IMFs) from empirical mode decomposition (EMD) applied over crude oil price data. Yang and Wang (2021) used EEMD into wavelet neural network to establish a hybrid neural network for predicting global energy prices. Another application of EMD was done by Fei (2015) in kurtosis forecasting of bearing vibration signal by establishing a hybrid model of EMD and RVM. EEMD has also been used by Wang et al. (2022) for data decomposition and to build a hybrid ensemble system with better forecasting performance over their data. Recently in the domain of water resource management, Ahmed et al. (2022) made use of CNN, BiLSTM and Ant Colony Optimization (ACO) for water level forecasting. The empirical mode decomposition with adaptive noise and variational mode decomposition were used in conjunction with the proposed model. The superiority of the proposed methodology was demonstrated using a case study. Through our study we aim to investigate the data characteristics of the streamflow in the Periyar. We further attempt to investigate the suitability of data decomposition and reconstruction based methods to this data using SARIMA models as well as deep learning models.

The contributions of the present research can be summed up in the following points.

In this study, we have developed and employed a novel methodology of creating different datasets, driven by data characteristics-based methodology. The different datasets, i.e. reconstructed components after ensemble empirical mode decomposition (EEMD), data without reconstruction, components after statistical decomposition, and the original raw data have been used for model building, analysis and drawing insights on forecasting. This issue was not explicitly explored in river flow time series.

Deep Learning (DL) techniques like LSTM, CNN BPNN, TCN and GRU are applied to predict all reconstructed components as well as individual decomposed components. Then we sum up all predicted values to ensemble them for final prediction. For benchmark and comparison purpose, we have done the entire methodology as said

above using traditional statistical models, i.e. SARIMA. Then a comparative analysis is carried out from the results of all the models for insights and further research prospects. Such comparisons with river flow time series data is not reported in the literature (to the best of authors' knowledge).

Although different performance metrics were proposed earlier, these were not classified in previous works. We have classified them into four types (Section 2.4) and demonstrated their usage.

Finally, we have demonstrated the application of the developed methodology elaborately with real data.

The previous research gaps, pros and cons of the proposed methodology are summarised in Table 1.

The rest of the paper is organized as follows. The entire methodology is described in Section 2. In Section 3, the details of the study area are presented; and Section 4 comprises of analysis, results and discussions. Section 5 is on model performance using data with missing values imputed followed by conclusions in Section 6.

2. Methodology

Fig. 1 describes the proposed methodology which comprises four stages. In stage 1, historical river flow data are collected and data characteristics analysis is done. By data characteristics we consider pattern and nature characteristics (Tang et al., 2013). Depending on the data characteristics, the necessity for data decomposition is assessed. In stage 2, data decomposition is performed. As time series data possess various pattern characteristics, time series decomposition is traditionally performed to reveal its trend (T_t), seasonal (S_t) and random (E_t) patterns. There are other methods of time series decomposition. Recent literatures focus on ensemble empirical mode decomposition (EEMD) method (Yu et al., 2015). EEMD generates several intrinsic mode functions (IMFs). Based on the similarity of their data characteristics, the IMFs may be aggregated to reconstruct component data (lesser in dimensions). Both EEMD based IMFs development and reconstruction are studied here. To have a comparative platform from data types point of view, the TSE decomposition is also done. In stage 3, first the component level prediction is exercised followed by ensemble prediction using sum activation function. Both TSE and EMD based components are individually predicted. In addition, the original time series (without decomposition) is also predicted. Finally, in stage 4, the performances of the prediction models are evaluated and compared using several performance measures and test statistics. Then, the key findings of this and the earlier stages are used to develop a decision support framework for effective management of river water. The following sub sections describe the steps in detail.

Table 1
Research gap and positioning of the present study.

Potential Gaps	Pros of our strategy	Cons of our strategy
• Lack of literature on application of decomposition-ensemble deep learning (DL) models on river flow forecasting.	• Proposed a methodology using decomposition-ensemble DL models with an application.	• Dataset from only a specific river was considered
• Data Characteristic Analysis (DCA) of river flows is absent in literature.	• DCA is considered in the proposed methodology.	
• Need exploration on how different intrinsic mode functions (IMFs) can effectively capture non-linear behaviour of complex river flows	• The proposed methodology can help in generating insights when subjected with different datasets and models.	
• Need exploration of decomposition-ensemble structure	• Both time series and EMD are considered.	
• Need more computational experiments	• The proposed methodology is flexible and amenable to variety of computational experiments	• Could be tested on various datasets

2.1. Data characteristics analysis

Tang et al. (2013) defines data characteristics as the special properties of data such as stationarity, linearity, seasonality, complexity, regularity, sudden change and randomness, which can be grouped under nature characteristics of data based on data generation process and the patterns characteristics of data that are governed by causal factors such as environmental and socio-economic factors.

2.1.1. Nature characteristics

The main nature characteristics affecting time series modelling are stationarity, nonlinearity and complexity. A general rule of stationarity means that a time series, say $x(t)$, and its distributed properties do not change over time. A time series $x(t)$ is strictly stationary if the joint distribution of $\{x_{t_1+\tau}\}$ and $\{x_{t_2+\tau}\}$ are same for all t_i and τ, where i = 1,2,3,….n and {} represents a set of n values (Huang et al., 1998). A widely accepted stationarity (also called weak stationarity) satisfies the following:

i) $E(x_t) = \mu$ (a constant)
ii) $E(x_t^2) < \infty$ (finite for all t)

$$Cov(x_{t_1}, x_{t_2}) = Cov\left(x_{t_1+\gamma},\ x_{t_2+\gamma}\right) = Cov(t_1 - t_2) \tag{1}$$

where Cov stands for covariance. Item (ii) above also satisfies that the variance of x_t, i.e., $E(x_t - \mu)^2$ is finite for all t.

Several statistical tests were proposed to test the stationarity of time series data such as Augmented Dicky-Fuller (ADF) test (Dickey and Fuller, 1981), Phillips Pierson (PP) test, KPSS test, Ljung-Box test, Dubin-Watson test, etc. (Phillips and Perron, 1988, Kwiatkowski et al., 1992, Ljung and Box, 1978, Durbin and Watson, 1950).

There exists significant change in modelling a non-linear process in comparison to a linear process. To understand whether a time series is generated from a non-linear dynamic process, *non-linearity* test is performed. The popular surrogate based methods in this regard are the weblet transform, fast Fourier transform (FFT), hidden Markov models, etc. Theiler et al. (1992) described a stationary approach using surrogate data for detecting non-linearity in time series. Their approach assumes a linear process of data generation as the null hypothesis (H_0) and then generated surrogate data with H_0. Then it compares the surrogate data with the original data using certain discriminating statistics such as correlation dimension, Lyapunov exponent and forecasting error. Unsworth et al. (2001) used Kolmogorov-Smirnoff (K-S) test for this purpose. If H_0 is rejected for a certain probability level (say, 5 %), the data possess *non-linearity*.

In Fourier transform (FT) based surrogate generating method, the squared amplitudes of the discrete FT, S_k ($k = 1, 2,\ldots, n$) is computed from the original data x_t (t = 1,2,..,n) (Tang et al., 2013), where

$$|S_k|^2 = \left|\frac{1}{\sqrt{n}}\sum_{t=1}^{n} x_t e^{j2\pi kt/n}\right|, k = 1, \ldots, n-1, n \tag{2}$$

Then, random phases $e^{j\phi_k}, \phi_k \in (-\pi, \pi)$ is multiplied with $|S_k|$ and then transform it back to the time domain, resulting into the surrogate data, $\widetilde{x}_t(t = 1, 2, \ldots, n)$ as below.

$$\widetilde{x_t} = \frac{1}{\sqrt{n}}\sum_{k=1}^{n}\left(e^{j\varnothing_k}|S_k|\right)e^{-j2\pi kt/n}, (t = 1, 2, \cdots, n) \tag{3}$$

Apart from surrogate measures, residuals from linear fit based methods and parametric methods are also used. Keenan (1985) provided

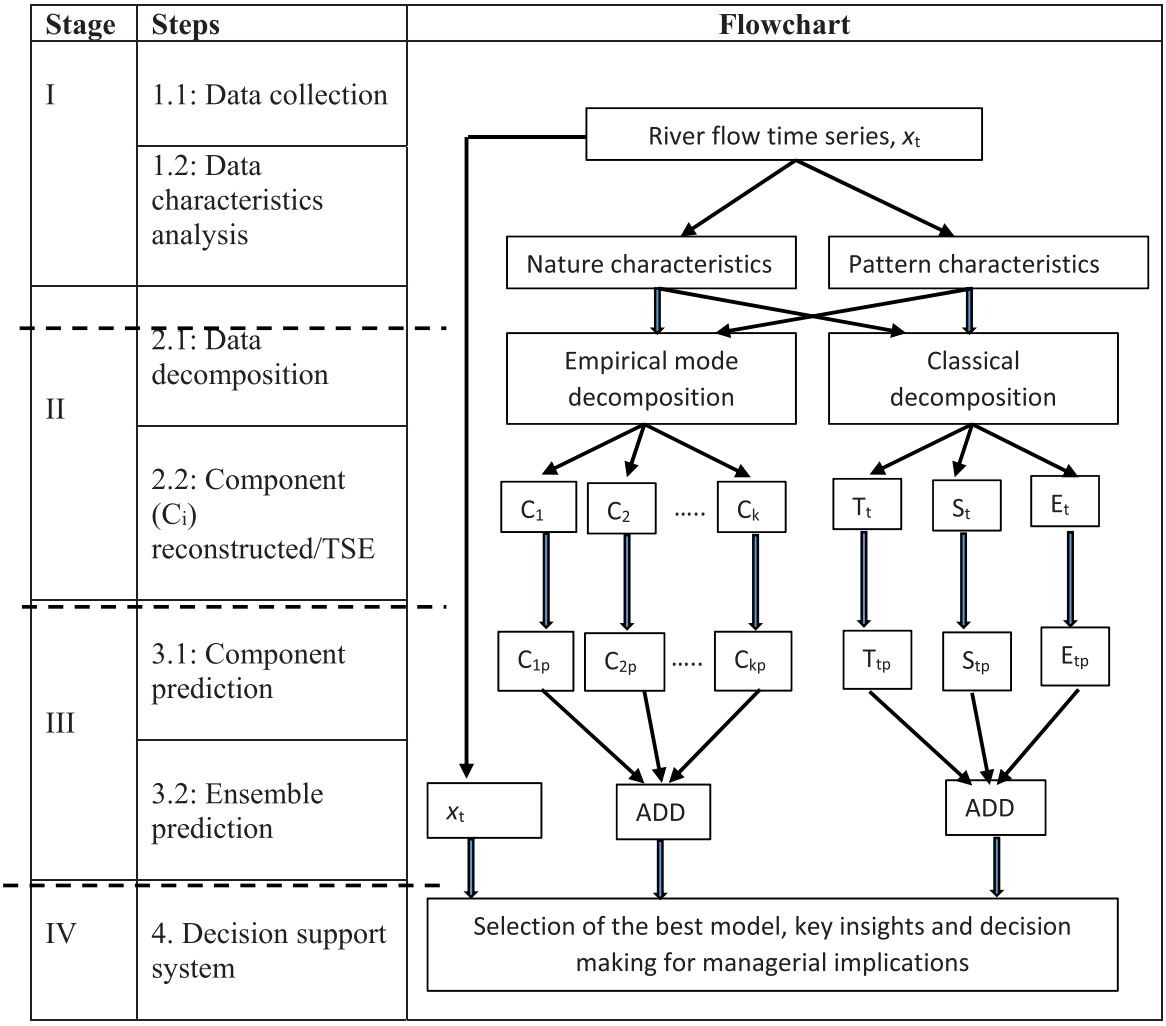


Fig. 1. Methodology for forecasting river flow time series.

the linear fit based methods. For a linear timeseries, AR(p) is sufficient to explain the data, where AR(p) is defined as

$$x_t = \mu + \sum_{j=1}^{p} \phi_j x_{t-j} + e_t \tag{4}$$

Note that in the above equation, ϕ_j, $j = 1,2, \ldots, p$ represents the first order linear regression coefficients of $x_{t\text{-}j}$, j = 1,2, …, p on x_t. It is devoid of higher order ($x^2_{t\text{-}j}$ or more) and interaction terms (say, $x_{t\text{-}j} \times x_{t\text{-}k}$, or higher terms). If the data generating process is non-linear, one or more higher order and/or interaction terms should be added to the right hand side of the AR(p) defined above. Now, if AR(p) defined above is fitted to a non-linear time series, the estimated residuals ($\hat{e}_t$, $t = p + 1, p + 2, \ldots, n$) will account for it, which may be implemented by regressing $\hat{e}_t$ on a proxy variable that represents the non-linear behaviour of the time series. Then, using F statistics, a test is made to determine the existence of non-linearity. Tsay (1986) modified Keenan's approach where Tuckey's one degree of freedom for non-additivity test is generalized to time series setting.

Complexity is an abstract term that includes many characteristics together such as chaotic property, memorability, irregularity, etc. It is often considered as a measure of the state between regularity and irregularity. Popular tests include correlation integral (Packard et al., 1980), permutation entropy (Bandt and Pompe, 2002), power spectral density, etc. Permutation entropy (PE) is preferred over others by many researchers as it is non-parametric, robust, and simple in computation. The steps to compute PE are given below.

Step-1: Partition the time series $\{x_t\}$, $t = 1, 2\ldots, n$ into embedding vectors of m embedded dimensions. With a time delay of τ, the embedding vectors can be defined as,

$$X_t = \{x_t, x_{t+\tau}, \ldots, x_{t+(m-1)\tau}\}, t = 1, \ldots, n-(m-1)\tau \tag{5}$$

Step-2: Find out the original patterns of x_t such that $\left[x_{t+(j_1-1)\tau} \leqslant x_{t+(j_2-1)\tau} \leqslant \ldots \leqslant x_{t+(j_m-1)\tau}\right]$, where j_k, $k = 1, 2,\ldots, n$, is the order of values $x_{t+(j_k-1)\tau}$ in the original embedding vector x_t. In case of tie, the order is maintained in terms of their appearance or by adding a white noise (random). For m embedding dimensions, there could be $m!$ order patterns (say, π).

Step-3: Compute the frequency for each pattern (π_i) as $p(\pi_i)$ where

$$p(\pi_i) = \frac{\#\left\{t \middle| t \leqslant n-m, \left(x_{t+(j_1-1)\tau}, x_{t+(j_m-1)\tau}\right) \text{ has type } \pi\right\}}{n-(m-1)\tau} \tag{6}$$

Step-4: Compute permutation entropy $H_p(m)$ and normalize $H_p(m)$. The permutation entropy of $H_p(m)$ of order $m \geqslant 2$ is defined as

$$H_p(m) = -\sum_{i=1}^{m!} p(\pi_i) \log p(\pi_i) \tag{7}$$

The normalized $H_p(m)$ is defined as

$$H_p = \frac{H_p(m)}{ln(m!)} \tag{8}$$

H_p lies between 0 and 1, the higher the H_p the higher the complexity of the time series.

2.1.2. Pattern characteristics

As stated earlier, the pattern characteristics relate to the patterns exhibited by time series data and are usually governed by environmental and economic factors. The patterns often exhibited by time series are trend-cycle (T_t), seasonality (S_t) and randomocity (E_t). So, considering data = patterns + Error, we may write

$$x_t = \left\{ \begin{array}{l} T_t + S_t + E_t, \text{ for additive model} \\ T_t \times S_t \times E_t, \text{ for multiplicative model} \end{array} \right\} \tag{9}$$

"Trend" (T_t) is defined as the systematic change in the level of a time series. The change could be both in the direction and slope (rate of change) of the series. When quantifying and/or exploring the trend is important, the researchers model it explicitly or otherwise remove it through a process called detrending. Trend can be linear or non-linear (polynomial) and is often captured through regression. "Seasonality" refers to the repeated patterns of increase or decrease in the time series that occur constantly throughout the duration of the time series (Jebb et al., 2015). Seasonality is usually related to calendar events and hence the total duration of repeating movements is often one year or less. Seasonal patterns may arise weekly, monthly or quarterly, etc. One of the striking feature of seasonality is that the seasonal patterns usually remain consistent over periods (Hyndman and Athanasopoulos, 2018). On the other hand, cyclicity refers to the non-seasonal recognisable pattern of the fluctuation over long periods. Seasonal pattern may be considered as a special case of cyclic patterns with one-year time scale (Tang et al., 2013). The methods used to determine cyclicity (including seasonality) are power spectral analysis, autocorrelation analysis, recurrence plots and R/S analysis (Kai et al., 2008). "Cyclic" (including seasonal) effects are often removed through seasonal adjustment. Alternatively, cyclic/seasonal effects can be modelled by regression with cyclical seasonal dummies (Chan and Cryer, 2008), or harmonic models (Cowpertwait and Metcalfe, 2009) or autoregressive integrated moving average (ARIMA) models.

In addition, the time series might have structural breaks known as "mutability". Tang et al. (2013) describes two approaches for testing structural change (mutability). In the first, a null hypothesis (H_0) is proposed as the time series doesn't have structural break against the alternative hypothesis (H_1) that at least one structural break exists in the time series. The popular test in this regard is chow test (Chow, 1960). The second approach is CUSUM type (Brown et al., 1975), which is based on cumulative sum of recursive residuals (CSRS). In this approach, an empirical fluctuation process (*efp*) is defined and its boundaries are computed under null hypothesis (H_0) of 'no structural change' with a probability of significance α. For mathematical details, see Brown et al. (1975). If the *efp*(t) crosses any of the boundaries, structural breaks exist at the points crossing the boundaries.

There exist significance tests for CSRS as given below (Zeileis, 2001).

$$S_r = \max_t \frac{efp(t)}{f(t)} \tag{10}$$

A thumb rule for most boundaries is $f(t) = 1$, but for recursive CUSUM, $f(t) = 1 + 2\,t$, $0 < t < 1$.

Chow's test (Chow, 1960) which uses F-test could also be used but efp based test is preferred over the F-test as the latter is applied for a single shift alternative. In addition to implement Chows's test one needs to know the breakpoints in advance. However, there is extension of Chow's test to calculate the F-statistics for all the potential change points (Zeileis, 2005) along with its boundaries and plotting. Further, there are several test statistics (namely supF, aveF, expF) available to objectively make decision about H_0.

Both the nature and pattern characteristics may be used to objectively define the overall complexity of the time series from computation point of view. Accordingly, simple or advanced forecasting techniques may be adopted for forecasting time series.

2.2. Data decomposition and component reconstruction

2.2.1. Classical time series decomposition

As stated in Section 2.1.2, a time series might have composed of several components. Using classical decomposition method, the series can be decomposed into trend-cycle (T_t), seasonal (S_t) and random (E_t) series. Two most popular approaches are additive model, where $x_t = T_t + S_t + E_t$ and multiplication model, where $x_t = T_t \times S_t \times E_t$. The additive model is used when the seasonal fluctuations do not vary with the level of the series. The multiplicative model is preferred when the seasonal fluctuations vary with the changes of the level of the series. The techniques used to decompose a time series are many. The popular ones are moving averages (MA), weighted moving averages (WMA), local regression smoothing (LRS) and seasonal-trend decomposition based on losses (STL).

2.2.2. Empirical mode decomposition

Huang et al. (1998) proposed EMD that decomposes the time series $\{x_t\}$ into several intrinsic mode functions (IMFs). An IMF represents the oscillating mode embedded in the data and is symmetric around local zero mean. The steps to extract IMFs from $\{x_t\}$ are given below (see Huang et al., 1998 for details). The process is known as sifting process.

Step 1: Consider $\{x_t\} = \{x_{t_1}, x_{t_2}, ..., x_{t_n}\}$, find out all local minima and local maxima, and develop the envelopes defined by the local minima (called the lower envelope) and by the local maxima (called the upper envelope). All data points must be covered within the envelopes.

Step 2: (a) Obtain the difference between the data, i.e., x_t and the mean μ_1 (of the envelopes). Say the difference is d_1, then

$$x_t - \mu_1 = d_1 \tag{11}$$

Check whether d_1 is an IMF or not. If d_1 satisfies the two requirements of an IMF described earlier, then d_1 is an IMF. Huang et al. (1998) pointed out that there could be over and undershoots and d_1 may not satisfy to become an IMF.

(b) Consider d_1 as the new data and repeat the steps 1 and 2(a), described above, which yields.

$$d_1 - \mu_{11} = d_{11} \tag{12}$$

where μ_{11} and d_{11} are mean and differences, respectively for d_1.

If d_{11} doesn't satisfy the requirements of an IMF, the sifting process continues till the kth step such that d_{1k} is an IMF, where

$$d_{1(k-1)} - \mu_{1k} = d_{1k} \tag{13}$$

So, d_{1k} is the first IMF and denote it as c_1 or $c_1 = d_{1k}$, subject to the stopping condition given below (c).

(c) To smoothen uneven amplitudes of the IMF so developed, Huang et al. (1998) had given a standard deviation (SD) based stopping criterion as below.

$$SD = \sum_{t=0}^{n} \left[\frac{|d_{1(k-1)}(t) - d_{1k}(t)|^2}{d^2_{1(k-1)}(t)} \right] \tag{14}$$

where, $0.2 \leqslant SD \leqslant 0.3$ is acceptable.

Step 3: Separate c_1 from x_t which yields residue r_1 as

$$x_t - c_1 = r_1 \tag{15}$$

Treat r_1 as the new data and repeat the steps 1 and 2, described above. Accordingly, the sifting process results into the following IMFs:

$$\begin{aligned} r_1 - c_2 &= r_2 \\ r_2 - c_3 &= r_3 \\ &\cdots \\ r_{k-1} - c_k &= r_k \end{aligned} \tag{16}$$

Step 4: Check c_k and r_k characteristics. As the final stopping criteria, either the c_k or r_k is very small below the threshold (such as SD) or the r_k becomes a monotonic function from which no more IMF could be extracted. The residue r_k usually represents a trend or a constant.

Step 5: Reconstruct the time series x_t again as

$$x_t = \sum_{i=1}^{k} c_i + r_k \tag{17}$$

What could be the total number of IMFs? Wu and Huang's (2009) recommended value is $\log_2 n$, where n is the length of the time series. Wu and Huang (2009) modified EMD by introducing a white noise to the original data. The modified method is named as ensemble EMD (EEMD) with a superior power of extracting signals (true IMFs).

2.2.3. Component reconstruction

The component reconstruction method based on EEMD was proposed by Yu et al. (2015). They consider the data characteristics driven (DCD) reconstruction following the work of Tang et al. (2013). In this study, we have adopted their approach for component reconstruction by combining similar IMFs in terms of their DCD scores. Yu et al. (2015) considered several data characteristics namely complexity (yes or no), mutability (yes or no) and tendency, and provided the following rules for component reconstruction.

Rule 1: If an IMF has high complexity, it should be considered as a single component irrespective of its values on other characteristics.

Rule 2: If an IMF is cyclic/seasonal, it should also be considered as a single component irrespective of its values on other characteristics.

Rule 3: If an IMF is non-cyclic/seasonal and has low level of complexity, it can be mixed with other similar IMFs based on their values on other characteristics (i.e. mutability and tendency).

2.3. Component prediction

Considering the latest literature trend, AI and machine learning based models provide superior results in comparison to traditional time series models. The application of ARIMA, ANN and LSSVR are quite common (Wang et al., 2011, Yu et al., 2015, Zhang, 2001, West and Dellana, 2011, Patel and Ramachandran, 2015). Recent literature in time series forecasting recommended the use of deep learning methods such as CNN and LSTM (Benidis et al., 2022, Wang and Duggasani, 2020, and Alsharef et al., 2022). In this study we investigated ARIMA, BPNN, CNN and LSTM.

2.3.1. Autoregressive integrated moving average (ARIMA) models

ARIMA model was popularized by Box and Jenkins in the 1970s and they have given a comprehensive methodology to implement ARIMA. ARIMA is a mixture of auto regression (AR) and moving averages (MA) together. If a time series is non-stationary, then differencing is used to make it stationary. Typically, a non-seasonal ARIMA model is termed as ARIMA (p, d, q) where p represents the order of auto regression, d represents the order of differencing, and q stands for the order of moving averages. If the time series x_t is stationary then $d = 0$ and ARIMA becomes ARMA with two component models together, i.e, AR(p) and MA (q) (Makridakis et al., 1998). Mathematically, ARIMA (p,d,q) is

$$\left(1 - \phi_1 B - ..... - \phi_p B^p\right)(1 - B)^d x_t = \mu + \left(1 - \theta_1 B - ..... - \theta_q B^q\right) e_t \tag{18}$$

where B is the backshift operator.

When seasonality is present, ARIMA is composed of two parts, non-seasonal ARIMA (p, d, q) and seasonal ARIMA(p,d,q)s, which are jointly represented as ARIMA(p,d,q)(P, D, Q)s. Here, s is the number of periods per season. For further details, readers may refer to Makridakis et al. (1998).

2.3.2. Back propagation neural networks (BPNN)

Artificial Neural Networks (ANN) is the most basic neural networks. Initially, they were built using the idea of mimicing the neuron structure of our brain. A neural network is composed of an architecture of layers. Each layer is made of an array of activation units or nodes. The input is fed to the first layer known as input layer and the final layer is called the output layer which gives the output. The layers between input and output layers are called hidden layers (Gopal, 2022).

Every layer is associated with a matrix of weights. A dot product of the input or activation unit vector and matrix of weights is carried out. Then, it is passed over an activation function which can be sigmoid, ReLu (Rectified Linear unit) or Lu (Linear unit) depending on the problem. The final output goes to the corresponding activation unit of the next layer. The input or training data set is fed into the input layer which is the first layer of the neural network. Then, the data is transmitted to the activation units of the next hidden layer using feedforward propagation. This step goes on until the last layer, also called the ouptut layer. If x_1,x_2,…,x_n represents the activation units of a layer and a_1 be the activation unit of next layer then the output transferred to a_1 is represented by the following equation (Gopal, 2022).

$$a_1 = f(W.X + b) \tag{19}$$

where f represents the activation function, W is the matrix of weights, X is the matrix of activation functions and b is the bias term. The BPNN architecture is shown in Fig. 2.

The weights are updated by the backpropagation algorithm. Using the training data, the actual and the expected output results are compared, iterated to numerically compute the partial derivatives and then optimal weights are obtained based on the optimization algorithm.

2.3.3. Convolutional neural network (CNN)

Convolutional Neural Network (CNN) is a deep learning algorithm known for its superior performance with image, speech and signal inputs. It consists of three types of layers, i.e., convolutional layer, pooling layer and fully-connected (FC) layer (Fig. 3). The convolutional layer is the core building block of a CNN. The components fed in it are the input data, a filter and it produces a feature map.

Let $X_{n\times n}$ represents the input matrix, $K_{1,j}$ denotes the first layer kernel. Then the feature map is (Gopal, 2022)

$$c_j' = \sigma\left(\left(X.K_{1j}'\right) + b_j'\right) \tag{20}$$

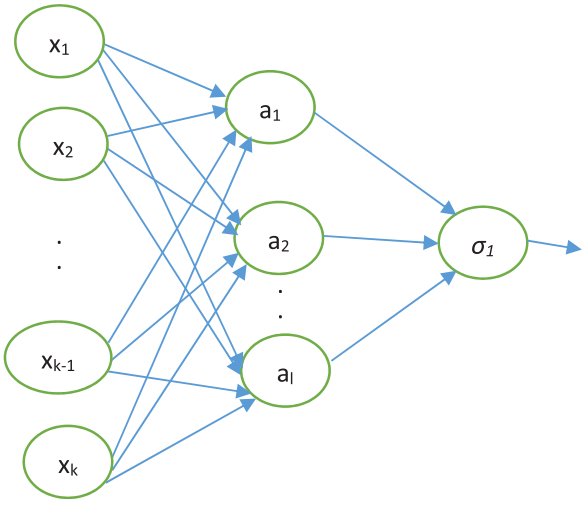


Fig. 2. BPNN architecture (with one output neuron).

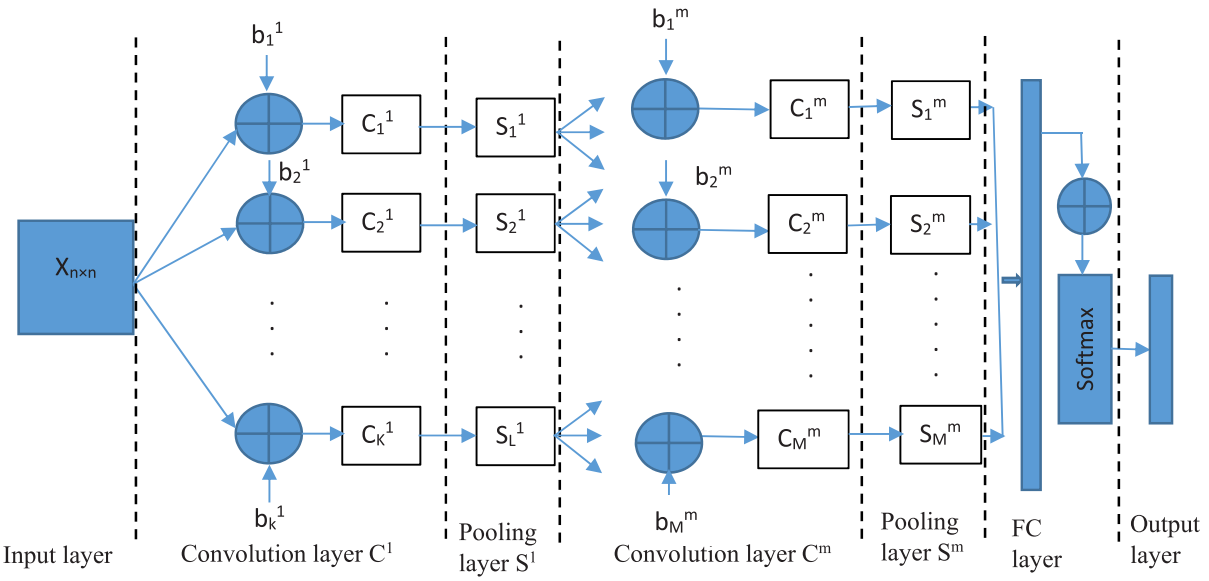


Fig. 3. CNN architecture.

where σ is the non-linear activation function. The commonly used activation function is ReLU. The pooling layer modifies the output c_j and provides summary statistics of it as output of the pooling layer, S_j. The size of S_j is usually lower than C_j. The most commonly used pooling is max-pooling. Other options are average-pooling and min-pooling. S_j, $j = 1,2,..,k$ is then fed to the second convolutional layer and S_l^2, $l = 1,2,..,L$ is produced following similar steps as done in the convolution + pooling layer 1. The process continues till the last pooling layer (say the M-th layer) is reached. The output at the M-th convolution layer is

$$c_j^M = \sigma\left(\sum_{i,j} S_i^M K_{ij}^M + b_j^M\right), j = 1, 2, \cdots\cdots, K. \tag{21}$$

which is modified as the M-th pooling layer and produces S_j^M, $j = 1,2,\ldots, K$.

The final layer of a CNN is a fully connected feedforward neural network which takes a vectorised input of the N number of S_j^M, j = 1,2, …, N. In the FC layer, softmax activation function is generally used.

A variant of CCN, namely temporal convolutional network (TCN) was developed (see Lea et al., 2016) to capture the time sequence dependencies by processing the entire dependence sequence instantly. TCN framework is adaptive and uses a hierarchy of causal and dilated convolutions.

2.3.4. Long short term memory (LSTM)

Long Short Term Memory (LSTM) is a type of Recurrent Neural Network (RNN). It was designed to solve the vanishing gradient problem in a standard RNN. Unlike the feedforward connections in traditional Artificial Neural Network (ANN), LSTM uses feedback connections and so is capable to processing entire sequence of data instead of treating each data points independently. LSTM is widely used in speech recognition, text and time series prediction. A typical LSTM architecture is shown in Fig. 4 (Gopal, 2022).

The key elements of LSTM are memory cells, and the three gates namely, forget gate, input gate, and output gate. The horizontal line through the top of the diagram is called the cell state. The cell state represents the information flow in the entire process. Information removal, retrieval and subsequent updation happens along this horizonal line which is regulated by the gates. Gates are composed of neural networks and pointwise multiplication operators, and they decide what

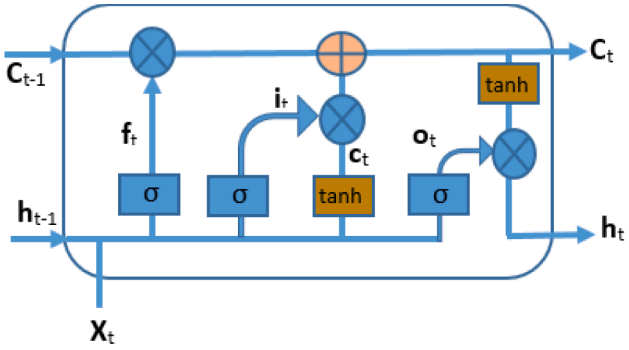


Fig. 4. LSTM architecture.

part of information should be rejected and what should be retrieved.

The forget gate layer is a sigmoid neural net layer which decides what information is to be rejected from the cell state. It receives input x_t and h_{t-1}. The output of such layer is a string of m (number of hidden neurons) between 0 and 1. If the output is 0 it means the location is irrelevant and can be removed completely; whereas an output of 1 mathematically means to completely keep the information. This is the first step of a LSTM module.

$$f_t = \sigma\left(W_f[h_{t-1}, x_t] + b_f\right) \tag{22}$$

where f_t is the output of the sigmoid layer, h_{t-1} is the hidden layer vector, x_t is the input vector, b_f is the bias vector and σ is the sigmoid activation function. The next step is to decide what information we are going to write to the memory cell. Input gate decides this. This happens in two steps. A sigmoid neural net decides what values to be updated. A *tanh* layer then creates a new vector C_t. Then we combine these two to update the old cell state C_{t-1} into new cell state C_t. Mathematically,

$$i_t = \sigma\left(W_i[h_{t-1}, x_t] + b_i\right)$$

$$\widetilde{c_t} = tanh\left(W_{\tilde{c}}[h_{t-1}, x_t] + b_{\tilde{c}}\right) \tag{23}$$

$$c_t = (f_t c_{t-1}) + (i_t \widetilde{c_t}) \tag{24}$$

Finally, the output gate computes the output as follows. A sigmoid neural net layer produces a bit vector mass O_t, which then multiplied by a tanh layer produces the output (h_t) of the LSTM unit (Gopal, 2022). Mathematically,

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * \tanh(C_t) \tag{25}$$

Gated neural network (GRU) (see Cho et al., 2014) is another variant of RNN and often considered to be an alternative of LSTM. GRU comprises reset gate along with current memory gate and update gate. The reset gate determines the amount of previous hidden state to be forgotten. The current memory gate ensures certain amount of non-linearity into the inputs. It also makes the input scale to zero mean. The update gate determines the amount of new inputs to be used to update the hidden state. Essentially, GRU has less parameters than LSTM and it is often assumed that GRU is faster than LSTM in case of less to moderate complexity of data sets.

2.4. Performance measures

There are several measures to evaluate the performance of prediction models. They can be broadly grouped as correlation based, index of agreement based (usually dimension less) and error based. On selecting suitable performance measures, one should consider (i) robustness of the measures in terms of applicability, (ii) commonly used and accepted in literature and (iii) specified strength in the model evaluation (Moriasi et al., 2007). Accordingly, we have considered the following performance measures for predicting adequacy and model comparison.

Let $\hat{x}_t$ represent the fitted/predicted values for the observed values x_t and N is the total number of observations considered for model evaluation. The correlation based measure, the coefficient for correlation 'r' is

$$r = \frac{\sum_{t=1}^{N}(x_t - \bar{x}) \times (\hat{x}_t - \bar{\hat{x}})}{\sqrt{\left[\sum_{t=1}^{N}(x_t - \bar{x})^2\right]^{1/2}\left[\sum_{t=1}^{N}(\hat{x}_t - \bar{\hat{x}})^2\right]^{1/2}}} \tag{26}$$

and $-1 \leqslant r \leqslant 1$. The higher the $|r|$, the better the model. However, r is overly sensitive to outliers and insensitive to additive and proportional differences.

The index of agreement (d) is (Willmott, 1981)

$$d = 1 - \frac{\sum_{t=1}^{N}(x_t - \hat{x}_t)^2}{\sum_{t=1}^{N}(|\hat{x}_t - \bar{x}| + |x_t - \bar{x}|)^2} \tag{27}$$

d lies between 0 and 1. d can accommodate/detect additive and proportional difference but highly sensitive to outliers. $d = 1$ for perfect agreement and is 0 for no agreement.

Nash-Sutcliffe efficiency (NSE) is a measure that reveals how will the observed and predicted data fit to 1:1 line (Nash and Sutcliffe, 1970).

$$NSE = 1 - \frac{\sum_{t=1}^{N}(x_t - \hat{x}_t)^2}{\sum_{t=1}^{N}(x_t - \bar{x})^2} \tag{28}$$

$-\infty < NSE \leqslant 1$. $0 \leqslant NSE \leqslant 1$ are acceptable. The following rules apply:

(i) NSE = 1 indicates 100 % match of predicted and observed data.
(ii) NSE = 0 indicates a model is no better than a mean model.
(iii) $-\infty < NSE < 0$ indicates worse fit than a mean model and hence, not acceptable.

The logical error based indices used are mean absolute percentage error (MAPE), root mean squared error (RMSE) and percent bias (PBIAS), where

$$MAPE = \frac{1}{N}\sum_{t=1}^{N}\left|\frac{x_t - \hat{x}_t}{x_t}\right| \tag{29}$$

$$RMSE = \sqrt{\frac{1}{N}\sum_{t=1}^{N}(x_t - \hat{x}_t)^2} \tag{30}$$

$$PBIAS = \frac{\sum_{t=1}^{N}(x_t - \hat{x}_t) \times 100}{\sum_{t=1}^{N} x_t} \tag{31}$$

The best values for MAPE, RMSE and PBIAS is zero. The lower the value the better the model predictions. Positive values of PBIAS indicate underestimation bias while the negative PBIAS indicates overestimation bias (Gupta et al, 1999).

To capture directional prediction accuracy, D_{stat} is used.

$$Dstat = \frac{1}{N}\sum_{t=1}^{N} d_t \tag{32}$$

$d_t = 1$ if $(x_{t+1} - x_t)(\widehat{x_{t+1}} - x_t) \geqslant 0$ or $d_t = 0$, otherwise.

A single measure cannot give a complete picture and each has its merits and demerits. However, these are the mostly used measures reported in literature.

As one of the objectives of this study is also to compare different competing models based on the performance measures (as stated above), it is better to have an objective test for comparison. One such test is Diebold-Mariano (DM) test (Francis and Roberto, 1995).

Let two competing models M_1 and M_2, let the forecast errors using test data are e_{1t} and e_{2t} where $e_{1t} = (\hat{x}_t - x_t)$ using M_1 and $e_{2t} = (\hat{x}_t - x_t)$ using M_2. DM assumes a loss function, $g(e_{it})$, for each forecast i ($i = 1,2$). $g(e_{it})$ is generally MSE or MAE based. Then, the loss differential (say d_t), $d_t = {e_{1t}}^2 - {e_{2t}}^2$ (based on MSE), or $d_t = |e_{1t}| - |e_{2t}|$ (based on MAE). DM assumes that the loss differential, d_t is covariance stationary and has short-memory. DM tests the following hypothesis:

H_0: Equal forecast accuracy for the two forecasts.

H_1: The forecast accuracies are different.

Under H_0, $E(d_t) = 0$. Let $\bar{d}$ be the sample mean of the loss differential. Then, the DM statistic is

$$DM = \frac{\bar{d}}{\sigma_{\bar{d}}} \sim N(0,1) \tag{33}$$

Here, $\bar{d} = \frac{1}{n}\sum_{t=1}^{n} d_t$ and $\sigma_{\bar{d}^2}$ is the consistent estimate of the variance of $\bar{d}$. If the two forecasts differ, then $|DM| > Z_{crit}$, where $Z_{crit} = Z(\alpha/2)$ for α probability level of significance.

The metrics can be grouped under four categories. Some are very generic metrics such as RMSE and MAPE and it is a standard practice to report them in any ML studies. These metrics primarily represent the accuracy in predicting the test data. We may consider them as Type-I metrics. The next category of metrics is domain specific such as index of agreement (d) and Nash-Sutcliffe efficiency (NSE). These metrics primarily focus on the acceptability of a prediction model in the domain of application. We may consider them as Type-II metrics. The third category is the directional prediction accuracy (Dsat). This reflects the ability of the model to hold the actual patterns (+ve or –ve) of relationships while predicting. We may call them as Type-III metrics.

The Type-II and Type-III metrics could be used in conjunction of the Type-I to enhance the confidence of a decision maker in choosing improvement/corrective actions. Although the above metrics could be used to compare the predictive models based on their computed and derived values, they generally do not provide objective statistical test for comparison. Hence, the fourth type of metrics (Type-IV), i.e., Diebold-Mariano (DM) test is used here. So, our recommendations for the use of these metrics are as follows:

1. First, use the common accuracy based (Type-I) metrics such as

RMSE and MAPE to assess the model fit from algorithmic performance point of view.

2. Then use the Type-II metrics to relate the performance of model from domain acceptability point of view.

3. If the absolute value of the Type-I and Type-II metrics give conflicting results or suffer with a lower value than desired, the D_{stat} (Type-III) metric could give a reasonable basis of using the findings or not. If D_{stat} is acceptable, use the findings for further actions with the intervention of domain experts.

4. Use the DM test (Type-IV metric) to specifically choose the best model among a set of alternative models.

3. Data

3.1. Study area

The Periyar is a west flowing river located in Kerala state in the Southern part of India. The Periyar basin spans an area of 5,398 km^2. The basin is situated between latitudes 9°15′30″N and 10°21′00″N and longitudes 76°08′38″E and 77°24′32″E. The Periyar originates in the Western Ghats and a considerable portion of the river's upstream catchment area is environmentally sensitive. The water storage and Inter-basin water transfer from the Periyar dam is a matter of concern between Tamil Nadu and Kerala. The downstream riparian district Ernakulam is a hub of Industrial activity and is densely populated. The Periyar is the longest river in Kerala and is the largest in terms of its discharge. Idukki dam, which is the largest hydroelectric project in Kerala state is situated upstream on the Periyar in Idukki district. The major tributaries of the Periyar are Muthirapuzha, Mullayar, Cheruthoni, Perinjankutti, Edamala, and Chalakkudy rivers. There are other minor tributaries including Chinnar, Cheruthony, Muthayar, etc. A map of the study area indicating the course of the river and the location of dams across the river is given in Fig. 5. To carry out the river flow forecasting, the daily time series data of Neeleswaram Hydrological Observation (HO) station for the period of 2017–2021 is utilized. In total we have 1263 observations (excluding 563 missing values), where each observation represents the past 10 year daily averages of the day concerned. From statistical point of view, the past 10 year daily average would be a better measure in comparison to single year daily average in terms of river flow forecasting. The Neeleswaram HO station is located in Ernakulam district towards the downstream stretch of the Periyar river. The location of this measuring station is indicated in Fig. 5.

3.2. Descriptive statistics

It has been observed that the observed data contain missing values. There are typically three kinds of missing values: (i) single missing value in a sequence, (ii) multiple missing values (a few) in a sequence, and (iii) large missing values. Hence, multiple imputations techniques were adopted. For (i), i.e., single missing value, we used linear imputation (i.e., the missing value is filled by the average of its preceding and succeeding observations). For (ii), rolling means were imputed. For (iii), the traditional imputations (like mean imputation) doesn't work. The seasonal-trend decomposition using LOESS (STL) based imputations are recommended. But the better approach could be to capture the time series patterns from the other years (say the preceding and succeeding years without large missing values) and impute with appropriate statistics. The daily average in the preceding and succeeding years is considered in this study. We have also done the STL based imputation. Interestingly, the daily average keeps the patterns better. Table 2 shows the descriptive statistics pertaining to the data. The minimum and maximum values remain same. There is a shift towards left in mean and the standard deviation is slightly lower when missing values are imputed.

3.3. Data characteristics analysis

We have considered three types of data namely overall river flow time series (Data set – I), classical decomposition-based components (T_t, S_t and E_t) (Data set – II) and IMFs (obtained through EEMD) based components (obtained after component reconstruction through DCD based mixing of IMFs) (Data set – III). Following the concepts described in Section 2.1, both pattern and nature characteristics of the three datasets were analysed. Figs. 6a & 6b show the original time series, x_t (top left panel) and the three components T_t, S_t, and E_t. The EMD based IMFs are shown in Figs. 7a & 7b. From the TSE plots (Figs. 6a & 6b), an

Table 2
Data Characteristics Analysis.

	Missing values deleted	Missing values imputed
n	1263	1826
Mean	212.44	186.82
Std. dev	191.78	178.73
Min	8.47	8.47
Max	1180.647	1180.647

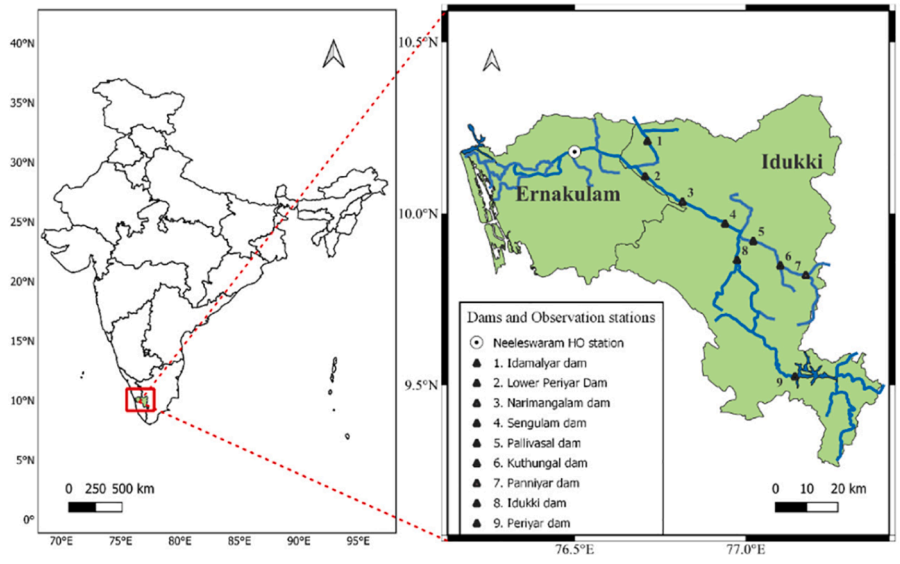


Fig. 5. Map of the study area.

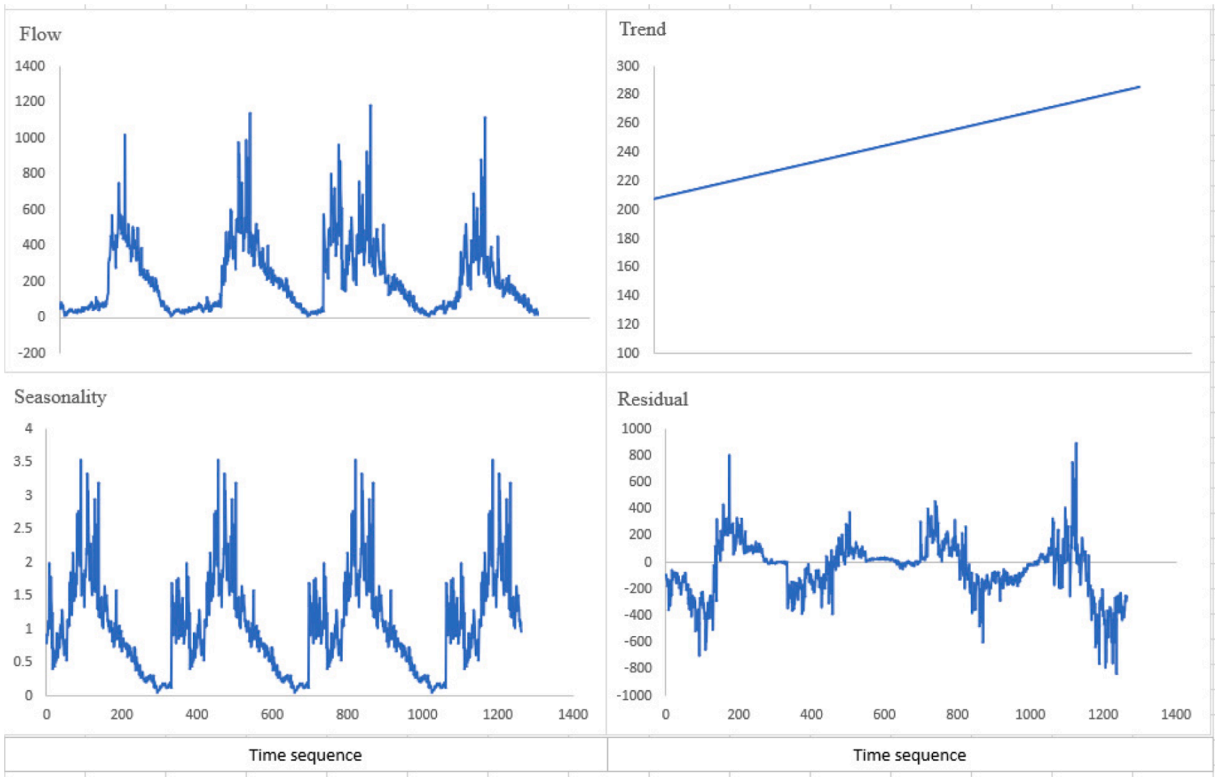


Fig. 6a. The original flow x_t (top left panel) and their T_t, S_t, E_t components (missing values deleted).

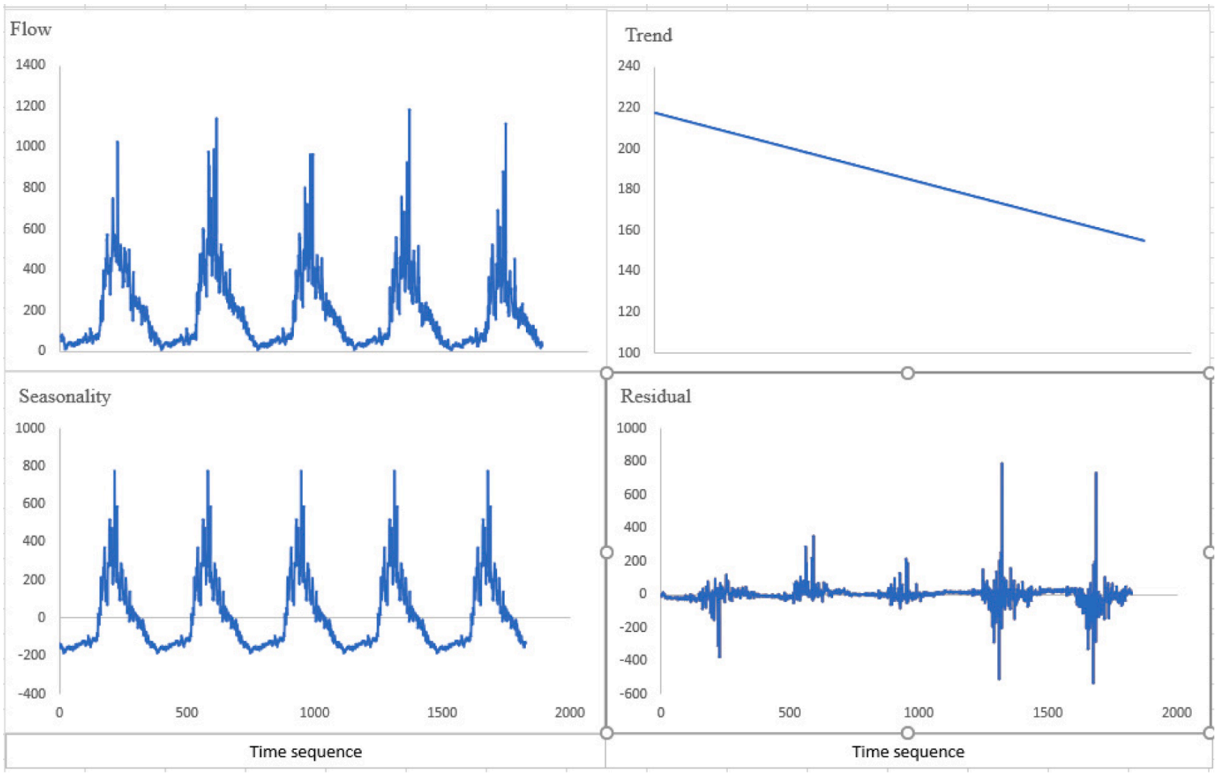


Fig. 6b. The original flow x_t (top left panel) and their T_t, S_t, E_t components (missing values imputation).

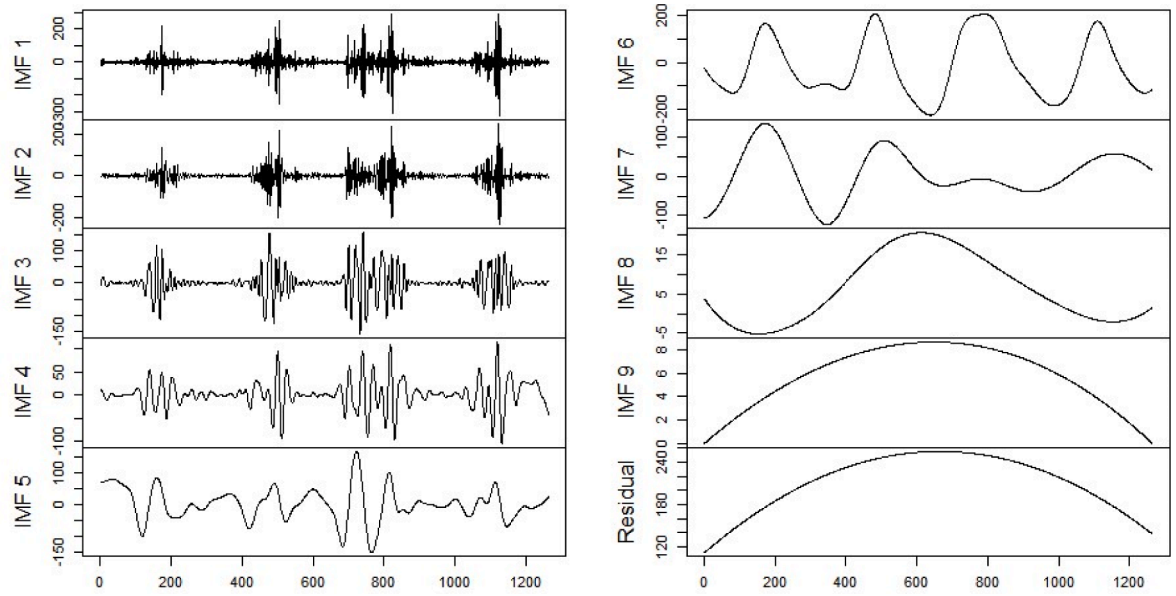


Fig. 7a. IMF components (IMF1 – IMF9 and residual) from EMD (missing data deleted).

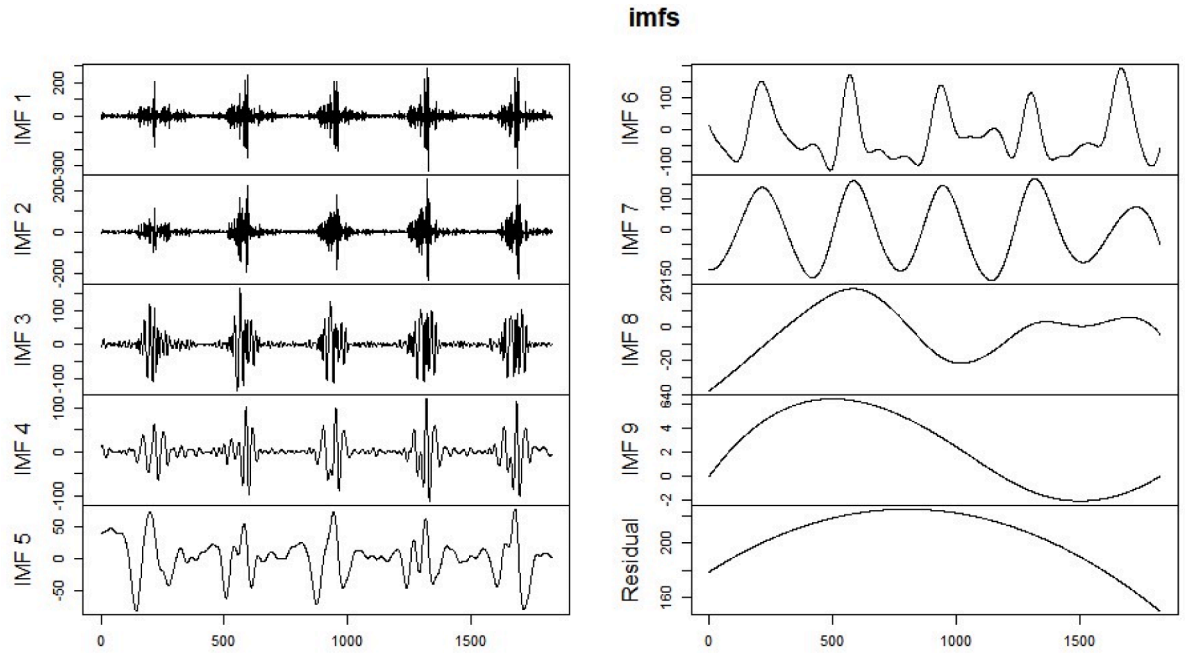


Fig. 7b. IMF components (IMF1 – IMF9 and residual) from EMD (missing data imputed).

interesting phenomenon in the trend plot is observed. There is a linear declined trend when missing data is deleted but the reverse is found for data with missing values imputed. As the river flow time series show similar annual cycles (see the overall flow plot, top left panel of Fig. 6), it is not clear why the trend behaves differently. However, the dominant components are the seasonal/cycle and the residual. But the EMD based decompositions (Figs. 7a & 7b) doesn't show any appreciable differences, rather, the intrinsic mode functions (IMFS 1–9 and residual) are almost alike.

From the data decomposition plots (Figs. 6a & 6b, 7a & 7b) and data characteristics study (Tables 3a & 3b), we observe the following key findings:

(i) The overall time series is a complex one. It is non-stationary (ADF = -3.39, $p < 0.05$), non-linear (KS test stastic (D) = 0.43, $p < 0.001$), seasonal, complex (PE = 0.91), and mutable (CUSUM test = 2.89, $p < 0.001$) with multiple break points. The main time scale seems to be 75 days (based on ACF plot).

(ii) The pattern characteristics analysis (see Fig. 6a) reveals that the time series has linear trend (T_t), harmonic seasonal (S_t) and complex residuals (E_t) components. The trend component shows a clear linear pattern. The dominant pattern is the seasonal component (S_t). The random component is apparently significant as fluctuating over a wide range. However, from the T_t S_t E_t decomposition, the complexity of S_t and E_t couldn't be deciphered from the visual plots. Similar patterns are observed for dominant components when missing data are imputed (Fig. 6b).

(iii) In Fig. 7, the time series plots for the IMFs are provided. The statistical analysis of these IMFs shows interesting results. IMF1 to IMF6 are non-stationary and the remaining components

Table 3a
Data Characteristics Analysis (missing data deleted).

Time series & comp.	Stationarity (ADF Test)	Non-linearity(Surrogate FFT (D))	Complexity (PE)	Maintime scale (days)	Breakpoints	CUSUM statistics
Original	−3.39**	0.49*	0.91	75 (230)	12	2.83*
IMF1	−11.63*	0.23*	0.91	2 (5)	25	0.15
IMF2	−15.96*	0.24*	0.68	3 (7)	NC	0.16
IMF3	−13.92*	0.23*	0.46	6 (15)	NC	0.39
IMF4	−9.37*	0.19*	0.3	9 (21)	11	0.39
IMF5	−6.77*	0.14*	0.18	21 (75)	16	3.53*
IMF6	−5.95*	0.23*	0.14	75 (240)	14	2.23*
IMF7	−2.69	0.26*	0.13	90 (260)	NC	6.65*
IMF8	−1.97	0.21*	0.12	256 (775)	4	5.38*
IMF9	−0.001	0.77*	0.11	278 (910)	4	12.80*
Residue	0.024	0.99*	0.11	283 (915)	2	13.25*

*Significant at 0.01 probability level and **Significant at 0.05 probability level, NC – not converged; Figures with () show 2nd sign reversal of ACFs.

Table 3b
Data Characteristics Analysis (missing data imputed).

Time series & comp.	Stationarity (ADF Test)	Non-linearity(Surrogate FFT (D))	Complexity (PE)	Maintime scale (days)	Breakpoints	CUSUM statistics
Original	−2.94	0.54*	0.91	82 (275)	17	3.26*
IMF1	−14.54*	0.23*	0.94	2 (5)	26	0.17
IMF2	−15.4*	0.24*	0.68	3 (7)	25	0.15
IMF3	−14.22*	0.26*	0.45	6 (12)	14	0.36
IMF4	−9.03*	021.*	0.3	10 (14)	NC	0.52
IMF5	−8.75*	0.22*	0.2	31 (100)	16	3.40*
IMF6	−5.76*	0.29*	0.16	67 (300)	15	2.33*
IMF7	−3.05	0.19*	0.13	95 (280)	18	4.72*
IMF8	−0.73	0.39*	0.11	243 (750)	9	11.79*
IMF9	−2.98	0.62*	0.11	520 (1500)	5	18.28*
Residue	0.03	1.00*	0.11	937 (1400)	1	15.93*

*Significant at 0.01 probability level and **Significant at 0.05 probability level, NC – not converged; Figures with () show 2nd sign reversal of ACFs.

including residual are stationary (based on ADF test). All components are non-linear. IMF1 is equally complex (PE = 0.91) as the original time series. The complexity of IMF2 is moderately high (PE = 0.67) and that for IMF3 is average (PE = 0.46) while others are much lower (with PE < 0.30). The IMFs have multiple breakpoints. The main time scale varies from component to component from a few days (for IMFs 1 & 2) to three quarter of a year (see for IMF8, IMF9 and residual). Similar patterns are observed when missing data are imputed (Fig. 7b).

While the classical (T_t, S_t, E_t) decomposition (Figs. 6a & 6b) gives the components that are physically interpretable, the EEMD based IMFs (Figs. 7a & 7b) are not easy to interpret physically. But from data complexity point of view, looking from IMF1 to IMF9 and residual, there is a gradual increase in smoothness. This implies that the complexity of the data decreases from IMF1 to residual. However, the classical decomposed components (S_t, E_t) do not have such order (except T_t, the trend component). Even the random component (E_t) looks to be complex data and is not perfectly random.

3.4. Component reconstruction

The data characteristics analysis of the IMFs and the residuals suggests for combining some of the IMFs together to generate newer components without sacrificing their overall complexity. Following the scheme provided in the Section 2.2.3, five components are reconstructed as follows:

- Following Rule 1, IMF1 and IMF2 are considered as separate components (with high & moderately high complexity). We termed them as C_1 and C_2, respectively.
- IMF3 and IMF4 have average complexity (0.30 < PE < 0.50), and lower main time scale. Hence, following Rule 2, they are grouped into one component as C_3.
- Similarly, IMF5 – IMF7 with low complexity (PE < 0.20) and medium main time scale are grouped into component 4 (C_4).
- Finally, following Rule 3, IMF8, IMF9 and residual are grouped as component 5 (C_5).

4. Empirical assessment

4.1. Modelling scheme

We are interested to investigate how deep learning (DL) algorithms such as BPNN, CNN, LSTM, GRU and TCN perform in predicting river flow time series. We assume that these models are better for complex time series in comparison to traditional statistical models. Using the concepts discussed in Section 2, we have assessed the overall complexity of the original river flow data. Considering the principle of EMD, we further assume that the individual IMFs will have overall complexity lower than the original data and some of the IMFs might represent simple data scheme (e.g., a linear process). Depending on the data characteristics, some of the IMFs may be aggregated to form EMD based component data where a component may be a single IMF or a combination of IMFs (reconstructed). We further hypothesized that the low complex components may be equally predicted by the statistical and DL models.

The DL model performance depends on many design parameters such as number of lags, batch size, dropout rate, number of epochs, and number of hidden layers. As river flow data has seasonal effects, which may be weekly, monthly or quarterly, we have chosen a lag of 7 days (weekly) as the other seasonal periods (monthly and quarterly) are multiples of a week. Further, weekly lag seems to be sufficient as in traditional SARIMA model. However, in this study, we run the models

(using overall data for different lags (of multiples of week). We have also varied other design parameters and selected the best possible architecture as per the experiments. The deep learning (DL) models' performance is further compared with benchmark models. Yu et al. (2015) considered ANN and LSSVR as benchmark models.

For parameter tuning of the DL models, an exhaustive sequential search was adopted. A base model (usually the default model) is computed first. Then, the epochs are varied and the best epoch is determined (following the minimum root mean squared residual (RMSE) value). Then, the base model is updated. The updated model is then used to evaluate the next hyper parameter (say, dropout) and the best value is determined using the minimum RMSE value. The process continues till all the hyper-parameters are tuned. As the process doesn't consider for all possible combinations of the hyper-parameters, finally, a few arbitrary random combinations are verified and their RMSE values are compared with the chosen model (based on the sequential search). Interestingly, no alternative best models could be found. Owing to a very large number of hyper-parameter combinations, the grid or random search method with all hyper-parameters was not adopted. This has also caused the programme to end prematurely. However, based on the sequential search, the search space was reduced and grid search was adopted for some models. As no appreciable improvement was found, we adopted the sequential search coupled with random checking. Fig. A1 (see Appendix A) shows the results of hyper-parameter tuning of LSTM.

For structuring our data, we chose a lag of 7 days. So for predicting the flow in the ith day, the corresponding input dataset is the flows on the $(i\text{-}1)^{th}$ day, $(i\text{-}2)^{th}$ day,… to the $(i\text{-}7)^{th}$ day. The SARIMA model was run using the pm.auto.arima programme which searches the best model using different combinations of (p, d, q)(P,D,Q)s.

Based on the sequential search, several BPNN models were run. For example, one comprises one hidden layer, 64 neurons, and the ReLU activation functions. The number of epochs was chosen at a higher level, i.e., 300 after running the model over a wide range of epochs. Another one comprises one hidden layer, 96 neurons, 0.15 dropout and the sigmoid activation function. The number of epochs (at a lower side) and batch size are 5, and 16, respectively.

The CNN model consists of a six layered CNN architecture with two convolutional layers, similar to LeNet-5 (LeCun et al., 1998). LeNet-5 was originally used for hand written digit prediction and the input vector data is similar to our time series data. Hence, it is one of the preferred models for time series forecasting. 64 filters of sizes 2×2 was applied with stride size of 1 to form the convolutional layers. Finally, it was trained with fully connected layers having 64 neurons. ReLU activation functions were used in the CNN model.

Similarly, other models were chosen. For example, the LSTM model comprises one hidden layer with 32 neurons using ReLU activation function. A dropout rate of 0.2 and batch size of 32 was adopted. The number of epochs was chosen at its best levels. The number of epochs of 30 appeared to be best; however, we have also used 300 epochs. Increasing the number epochs doesn't improve the model performance. Please see Fig. A1 (in Appendix A) for the plots.

4.2. Competing models

Following the methodology (in Section 2) and modelling scheme above (in Section 4.1), the river flow time series was analysed. The data characteristics of the time series were computed using R; classical decomposition was done using MINITAB, intrinsic mode functions (IMFs) were obtained using R, and the forecasting models were computed using Python. Because of the similarity in characteristics of the dominant components in classical decomposition and similarity in characteristics of intrinsic mode functions (IMFs) based on EMD decomposition for both the cases, i.e., with missing data deleted and missing data imputed, and as the missingness mechanisms are not known in advance, first, we utilized the available data (missing data deleted) and performed the modelling as outlined in Fig. 1.

Firstly, we have considered four models M1 to M4, namely seasonal ARIMA (SARIMA), backpropagation neural networks (BPNN), convolutional neural networks (CNN) and long short-term memory (LSTM), respectively. As stated earlier, three data sets were used. They are the time series (x_t), classically decomposed components (T_t, S_t and E_t) and EMD based components reconstructed from IMFs (we refer it as C_1 to C_5). So, the four forecasting models (M1 – M4) in combination with the three datasets, provided us with 12 competing models. We name them as $\{M_{11}, M_{12}, M_{13}\}$, $\{M_{21}, M_{22}, M_{23}\}$, $\{M_{31}, M_{32}, M_{33}\}$ and $\{M_{41}, M_{42}, M_{43}\}$. From model building and validation point of view, we have adopted machine learning approach of splitting datasets into a training set (with the first 70 % of series) and test set (with the remaining 30 % of the series). Each of the model (M_{11} to M_{43}) with its consequent component data series were trained using the respective training set. It is to be noted here that the models M_{11}, M_{21}, M_{31}, M_{41} were trained using the original river flow series (x_t) training data set (one series). The models M_{12}, M_{22}, M_{32}, M_{42} have three components (T_t, S_t, E_t) and each component models are trained separately using the concerned component data (See Tables 3-7). Similarly, each one of M_{13}, M_{23}, M_{33}, M_{43} has 5 component models and were trained using respective component data (C_j, $j = 1, 2\ldots,5$). So, each of the forecasting models was trained separately using 9 data sets (1 original, 3 TSE and 5 EMD). For the four forecasting models, we trained 36 component models (note: the original data model is one component model). After training each of the models (using its components), each was used to predict the future series. For component prediction models (using 3 component TSE and 5 EMD), the predicted components are added up to predict the overall time series, i.e. $\widehat{x_{t+1}}, \widehat{x_{t+2}}, \ldots, \widehat{x_{t+k}}$.

$$\widehat{x_{t+k}} = \widehat{T_{t+k}} + \widehat{S_{t+k}} + \widehat{E_{t+k}} \text{ for } 3 - \text{component TSE based models, and} \tag{34}$$

$$\widehat{x_{t+k}} = \sum_{j=1}^{5} \widehat{c_{j+k}} \text{ for } 5 - \text{component EMD based models} \tag{35}$$

where $k = 1, 2\ldots$, length of test data.

It is to be noted here that model respecifications might be needed based on models' performance evaluation.

4.3. Model evaluation

The original test data set was used to validate the models forecasting performance. We have considered seven performance measures. They are coefficient of correlation (r), index of agreement (d), Nash-Sutcliffe efficiency (NSE), mean absolute percentage error (MAPE), root mean squared error (RMSE), percentage bias (PBIAS), and directional prediction accuracy (D_{stat}). See Section 2.4 for details.

Using these seven performance measures, we assess the performance of each of the four forecasting models (M_1, M_2, M_3 and M_4) under the four heads:

(i) Performance of a model with different datasets: Here we treated each forecasting model (e.g. BPNN) as a standalone model and we assume that the model performance varies with data characteristics following the insights from data characteristics literature (see Section 2.1).
(ii) Performance of forecasting models with same data set: Here we assume that because of the contexts of development and assumptions underlying a model, different models will perform differently with the same data. We further assume that a model with better performance (in terms of the seven performance measures) better mimics the data generation process.
(iii) Performance of forecasting models with different datasets: Here we assume that some models are more robust than others. Hence,

Table 4
Performance measures of SARIMA.

Model (M_1)	M_{11}: x_t	M_{12} $x_t=T_t+S_t+E_t$	M_{13}:$x_t = \sum_{i=1}^{5} C_i$	Remarks
	Dataset − I	Dataset − II	Dataset − III	
r	0.41	0.41	–	Poor fit
d	0.34	0.32	0.19	Poor fit
NSE	−6.90	−8.64	−0.07	Worse fit than a mean model
MAPE	220.05	213.06	458.12	Moderate fit
RMSE	155.66	153.73	657.14	Moderate for M_{11} and M_{12}, and high for M_{13}.
PBIAS	–32.45	−28.76	303.01	Underestimation bias for M_{11} and M_{12}, and overestimation bias for M_{13}.
D_{stat}	0.55	0.55	0.54	55% agreement
Overall comments	SARIMA is not a good fit for the data sets.			

Table 5
Performance measures of BPNN.

Model (M_2)	M_{21}: x_t	M_{22} $x_t = T_t + S_t + E_t$	M_{23}:$x_t = \sum_{i=1}^{5} C_i$	Remarks
	Dataset − I	Dataset − II	Dataset − III	
r	0.92	0.90	0.85	Very good fit
d	0.96	0.89	0.83	Very good fit
NSE	0.84	0.56	0.43	Very good for dataset I, moderate for dataset II, poor for dataset III
MAPE	25.30	49.10	41.07	Very good
RMSE	59.53	90.09	113.91	Very good
PBIAS	0.82	−2.56	−4.18	Very good for dataset I, slight overestimation noted for dataset II & III
D_{stat}	0.63	0.58	0.58	Moderate fit
Overall comments	Very good for all datasets but the best results are obtained with original data series. Component ensembles are not better than total data model. BPNN performs better with original time series.			

Table 6
Performance measures of CNN.

Model (M_3)	M_{31}: x_t	M_{32} $x_t = T_t + S_t + E_t$	M_{33}:$x_t = \sum_{i=1}^{5} C_i$	Remarks
	Dataset − I	Dataset − II	Dataset − III	
r	0.84	0.81	0.84	Good fit
d	0.82	0.80	0.83	Good fit
NSE	0.32	0.40	−0.40	M_{33} performs worse than mean model
MAPE	34.32	67.33	41.15	Good fit
RMSE	110.03	134.35	113.31	Good fit
PBIAS	10.96	−2.24	3.86	Underestimation bias for I, III and overestimation bias for II.
D_{stat}	0.63	0.59	0.61	Moderate
Overall comments	M_{32} is better in terms of NSE while M_{31} has lowest RMSE and MAPE. M_{33} performs worse that mean model.			

Table 7
Performance measures of LSTM.

Model (M_4)	M_{41}: x_t	M_{42} $x_t = T_t + S_t + E_t$	M_{43}:$x_t = \sum_{i=1}^{5} C_i$	Remarks
	Dataset − I	Dataset − II	Dataset − III	
r	0.87	0.87	0.83	Good fit
d	0.85	0.86	0.82	Good fit
NSE	0.38	0.44	0.37	Moderate fit for all models. M_{42} is most efficient.
MAPE	0.47	0.55	0.39	Average for M_{42}, others poor
RMSE	97.52	97.96	114.10	Good fit
PBIAS	−1.99	−5.24	6.33	Good fit with a little overestimation for M_{41}, M_{42}. Underestimation bias for M_{43}.
D_{stat}	0.65	0.62	0.60	Moderate
Overall comments	M_{42} is the best model in terms of NSE. RMSE and MAPE for all models are comparable.			

we will assess how each of the models efficiently handle different data sets.

(iv) Final comparison of the models: Here we do selection of the best models among the alternatives. This derives the final conclusions arising out of the assessments (i), (ii) and (iii) stated above.

The performance statistics are shown in Tables 4-7 and Fig. 8. From assessment (i), SARIMA model provides the worst fit. For all the three datasets (I, II, III), NSE statistics value is negative, which indicates that a model is worse than a mean model. However, none of the other statistics shows a very good fit to the data. From RMSE point of view, M_{13} (SARIMA with EMD components) is the worst (with RMSE = 657.14) in comparison to other SARIMA models (i.e, M_{11} and M_{12} with RMSE 155.66 and 153.73 respectively). The three BPNN (M_2) models (M_{21}, M_{22} and M_{23}) provide a very good fit to all three datasets (I, II and III). The coefficient of correlation (r) and index of agreement are very high (0.89 or above) except for M_{23} (with $r = 0.85$ and $d = 0.83$; which is still a good fit). The r value for model M_{21} is 0.92 indicating a perfect match.

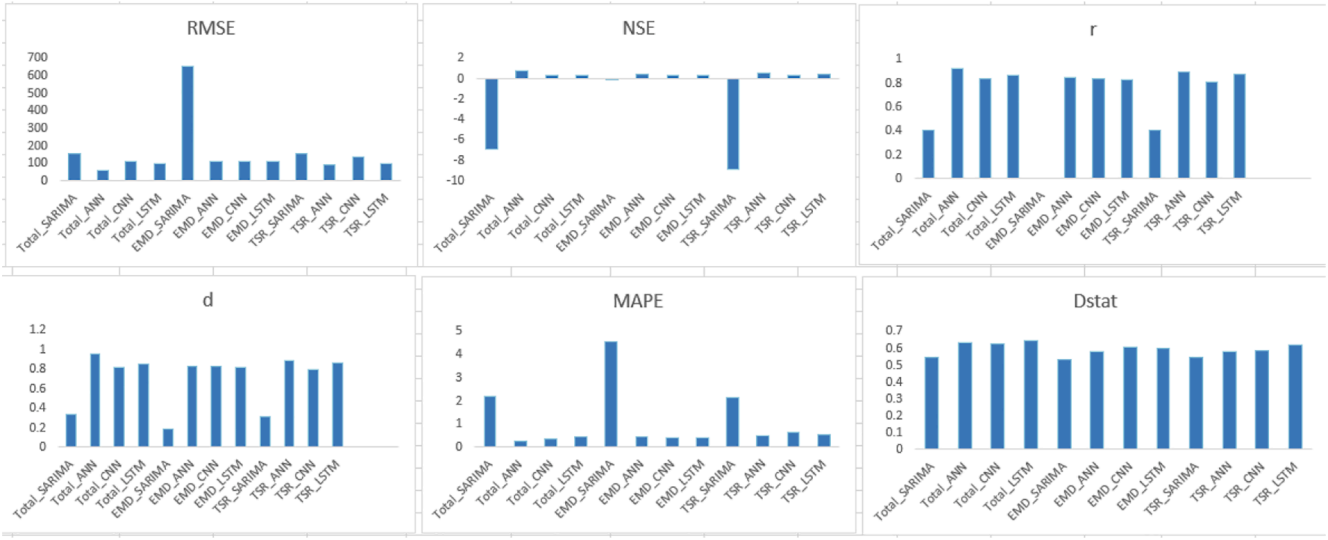


Fig. 8. Performance statistics.

The RMSE value of M_{21} is also low in comparison to M_{22} and M_{23}. So, the BPNN best model is M_{21}. However, DM test will confer it (will be discussed later). The CNN (M_3) and LSTM (M_4) models are also good fit to the data with M_{33} and M_{42} being the best models for CNN and LSTM, respectively. Interestingly both CNN and LSTM based models do not perform better than the BPNN models.

From assessment (ii), the BPNN, CNN and LSTM based models provide equivalent results for each of the data sets (I, II and III) with a slight advantage to BPNN. From assessment (iii) point of view, BPNN models are found to be more robust than the other models. The unanswered question is why the "advanced" deep learning models like CNN and LSTM did not perform better than BPNN? Both CNN and LSTM are based on BPNN and are known to perform better for image data and natural language processing (NLP) respectively. The river flow time series is neither fit to image analysis nor to NLP; rather being numerical in values is well suited for BPNN applications. The added complexity of CNN and LSTM models doesn't provide additional advantages (often expected). But readers should not take this finding overwhelming as we have used only one river flow time series. The same should be validated with more river flow time series from different countries.

Another interesting finding in this study is that the ANN based models (BPNN, CNN and LSTM) outperforms the traditional statistical model SARIMA. In fact, the best fit SARIMA is not a fit model to all the three data sets (I, II and III). This is also in line with the findings from recent research in different domains of applications (Patel and Ramachandran, 2015; Benidis et al., 2022).

From the data characteristics point of view, the original river flow series is more complex than its components (either TSE or IMFs). The data characteristics study showed this. So, based on this assumption, component predictions should have better performance indices. Interestingly, the same proved to be true as we have seen better values in the component level prediction (results are not shown in this paper). For example, the RMSE values for the EMD based five components using LSTM are 62.55, 54.15, 76.26, 30.46 and 1.33 which are much less than the overall RMSE (114.09). This is perhaps due to the use of ADD aggregation of component level predictions to generate the overall

Table 8
DM test.

	DM test statistic values										
	M11	M12	M13	M21	M22	M23	M31	M32	M33	M41	M42
M12	**5.90**										
M13	**-16.43**	**-16.46**									
M21	**5.14**	**4.87**	**16.68**								
M22	**4.20**	**3.96**	**16.64**	-1.23							
M23	**5.79**	**5.52**	**16.75**	1.43	1.74						
M31	**6.48**	**6.18**	**16.77**	0.98	1.42	-0.74					
M32	**-7.85**	**-7.89**	**6.70**	**-8.72**	**-8.74**	**-8.72**	**-8.55**				
M33	**4.41**	**4.18**	**16.61**	0.61	1.04	-0.82	-0.35	**8.66**			
M41	**10.95**	**10.39**	**16.84**	2.94	3.19	0.47	2.11	**8.61**	1.66		
M42	**-10.79**	**-10.81**	**-4.40**	**-11.09**	**-11.09**	**-11.14**	**-11.07**	**-11.79**	**-11.10**	**-11.06**	
M43	**4.66**	**4.42**	**16.65**	0.62	1.03	-0.85	-0.40	**8.62**	-0.02	-1.71	**11.09**
	Bold faced values are significant at 0.01 probability level										

prediction. It is suggested that adding up of the component level predictions may not be a good idea; rather other activation functions could be used. However, the 'ADD' activation is a popular choice in recent literature (Yu et al., 2015).

4.4. Model comparisons

4.4.1. DM test

In order to select the best models among the set of alternatives, we have used Diebold-Mariano (DM) test. The results are shown in Table 8. The key findings from the DM-test are as follows:

a) The three ANN based models (BPNN, CNN and LSTM) clearly differ from SARIMA. SARIMA models for all the three data types are not fit to the respective data sets (see Table 3).
b) Among the ANN based models, mixed results are found using different data sets. For example, the predicted errors do not differ among the BPNN models using the three data sets. But the same is not true for CNN and LSTM based models.
c) There is no difference in the predicted errors for the BPNN, CNN and LSTM models using both original time series and EEMD based prediction.
d) There is difference in the predicted errors for the CNN and LSTM models in TSE component-based prediction and EEMD component-based prediction. The results favour EEMD component-based prediction.

4.4.2. k-fold cross validation

The performance of the DL models was also compared using k-fold cross validation (Tables 9 & 10). The models considered are BPNN, LSTM, GRU and TCN. The temporal convolution network (TCN) is a variant of CNN developed to capture the sequence dependencies (such as auto-correlation in timeseries data). The gated recurrent unit (GRU) is a variant of RNN and is often considered as an alternative to LSTM. CNN is not considered in k-fold cross validation as the TCN, developed for sequence dependence data, is considered here.

The seven performance measures (see Section 2.4) are used to assess the model performance. Table 9 demonstrates the comparison of the models using training data, while Table 10 shows the same for the test data. The k-value is considered as 5. James et al (2023) recommended the value of k as 5 or 7 as these values serve the purpose in most of the real-life applications. However, in this study k = 5 provides better performance than k = 7.

All the DL models fit the data well. So, for the river flow time series, the DL models are able to predict the future behaviour effectively. This is an interesting finding as the classical BPNN could also effectively predict the river flow pattern. However, considering the characteristics of the models and their use of applications during their development, the LSTM, GRU and TCN were specifically designed for sequence data. As these (time) sequence models are also performed excellently, their use may be preferred if the data complexity increases.

4.4.3. Comparison of the best models with Prophet

The Prophet model is a non-linear regression forecasting model with the following form (Hyndman and Athanasopoulos, 2018).

$$y_t = g(t) + s(t) + h(t) + e_t \tag{36}$$

Here $g(t)$ is called the growth term which captures piecewise linear trends. $s(t)$ captures the seasonal patterns and $h(t)$ capture the holiday effects. e_t is a white noise error term. The Prophet model shows promising results in various applications such as forecasting of disease outbreaks (Satrio et al., 2021), bitcoin values (Yenidoğan et al., 2018), and electricity demand (Bashir et al., 2022).

As it has been seen from Section 4.3 (Tables 4 – 7) that the model performances using the Dataset-I are superior than the decomposition-based datasets (II & III), the Prophet model was fitted on the Dataset-I (overall flow). Table 11 shows the performances of the Prophet. It is observed that the DL models outperform the Prophet.

5. Model performance using data with missing values imputed

As missing data often creates problems in prediction and it can lead to erroneous decision making, we attempted to explore the performances of the DL models when the missing data are imputed. As discussed in Section 3.2, depending on the types of the missing values, various imputation techniques (e.g, linear imputation, STL based imputation) were adopted to impute the missing values. We evaluate and compare the DL model performances using the overall river flow times series with missing data imputation.

The results of the data characteristics analysis (DCA) is shown in Table 3b. Interestingly, the DCA reveals that there are no appreciable differences in the nature and patterns when we compare with the DCA results of the Dataset-I (with missing data deleted). Readers may compare Tables 3a and 3b to see the similarity. This is an interesting finding as missing data imputation may change the nature of the data (i.e., may not be able to capture the true data generating mechanism). Fortunately, in this study no appreciable changes are found.

An exhaustive sequential search, as outlined in Section 4.1, was used for hyper-parameter tuning and selecting the best parameter combinations for each of the DL models. Please see Fig. A2 (in Appendix) for the hyper-parameter tuning of LSTM. Similarly, other model architectures were chosen for its best combination.

The DL model results using 5-fold cross validation are shown in Tables 12 and 13 for the training and test data, respectively. All the models show an excellent fit (based on the performance measures), and are slightly better than these models applied to the data with missing values deleted (see Section 4.4.2). Tables 12 & 13 show the performance measures computed and their 95 % confidence intervals (CI) for both the training and test data, respectively. It is to be noted that the 95 % CI was computed using t-distribution. It is conservative one as the actual distribution of the performance measures (e.g., RMSE, d, NSE) are often not known or too complex to use for practical purposes. However, the t-distribution based CI gives results with reasonable accuracy.

Table 9
Training data: 5-fold cross validation results (with 95 % CI) [data with missing values deleted).

Measures	ANN		LSTM		GRU		TCN	
	Statistic	95 % CI	Statistic	95 % CI	Statistic	95 % CI	Statistic	95 % CI
r	0.92	0.90 – 0.95	0.93	0.91 – 0.95	0.93	0.91 – 0.95	0.95	0.92 – 0.99
d	0.91	0.82 – 0.99	0.96	0.95 – 0.97	0.96	0.94 – 0.97	0.27	0.04 – 0.49
NSE	0.76	0.64 – 0.88	0.87	0.83 – 0.90	0.85	0.81 – 0.90	0.74	0.44 – 1.04
MAPE	67.69	0.013 – 1.25	29.64	0.21 – 0.38	38.38	0.18 – 0.59	31.91	0.13 – 0.51
RMSE	94.36	65.75 – 122.97	69.80	53.80 – 85.81	73.51	54.86 – 92.16	87.22	27.21 – 147.24
PBIAS	0.88	−18.13 – 19.88	0.15	−0.42 – 0.72	−2.37	−10.07 – 5.33	−26.75	−49.29 – −4.22
dstat	0.64	0.47 – 0.80	0.78	0.74 – 0.82	0.74	0.67 – 0.81	0.47	0.16 – 0.78

Table 10
Test data: 5-fold cross validation results (with 95 % CI) [data with missing values deleted).

Measures	ANN		LSTM		GRU		TCN	
	Statistic	95 % CI	Statistic	95 % CI	Statistic	95 % CI	Statistic	95 % CI
r	0.86	0.75 – 0.96	0.83	0.71– 0.96	0.86	0.75 – 0.97	0.85	0.74 – 0.95
d	0.82	0.74 – 0.89	0.90	0.83 – 0.96	0.90	0.85 – 0.95	0.24	0.11 – 0.38
NSE	0.44	0.20 – 0.67	0.68	0.49 – 0.87	0.69	0.55 – 0.83	0.56	0.23 – 0.90
MAPE	0.78	0.006 – 1.55	39.34	20.50 – 58.17	45.50	13.89 – 77.10	113.67	16.28 – 211.05
RMSE	106.27	61.25 – 151.29	89.35	22.79 – 155.91	86.58	28.15 – 145.02	101.72	12.44 – 191.00
PBIAS	−16.75	−62.20 – 28.69	−1.36	−7.74 – 5.02	−8.54	−29.25 – 12.18	−24.20	−37.82 – −10.59
dstat	0.62	0.50 – 0.75	0.73	0.67 – 0.78	0.72	0.64 – 0.81	0.28	0.08 – 0.49

Table 11
Performance measures for Prophet model.

Measures	Statistic
r	0.17
d	0.48
NSE	−0.91
MAPE	1.22
RMSE	204.64
PBIAS	13.18
D_{stat}	0.49

6. Conclusions

River flow forecasting is an important activity in water resource policy making. In the context of the Periyar river basin which is a flood prone area (affected by heavy floods in 2018 and 2019 that caught international attention) (Hunt and Menon, 2020; Singh and Chinnasamy, 2021; Vijayan et al., 2021), accurate forecasts can help in flood damage control and relief activities. Also, river flow forecasts help in management of water scarcity and can improve the quality of eco-system services and level of economic activity. Thus, development of forecasting models with high accuracy is an important challenge in hydrology and water resources management. Our study sheds light on the capability of decomposition techniques when used in conjunction with machine learning and deep learning techniques. The deep learning-based forecasting models in the present study can help to predict flow volume in the Periyar river basin thereby giving the signal of possible flood occurrences in the future. This can in turn aid the policy makers to device and implement policies for flood management and, rescue and relief activities effectively and efficiently. To the best knowledge of the authors, there are no existing forecasting models specifically developed for flow prediction for Periyar river basin and the present study is the first of its kind in this context.

It is observed that deep learning models decisively outperform traditional statistical models. The statistical models were not able to capture the dynamic nonlinear pattern of daily river-flow data. Several noteworthy outcomes of the study are as below.

1. Data characteristics study provided interesting insights regarding the river flow data at Neeleswaram Hydrological Observation station. Both the nature and pattern characteristics give valuable insights about the data generation process, of particular interests are complexity (using permutation entropy) and mutability (structural breaks).
2. The deep learning models outperform the SARIMA model by yielding lower error and better NSE values. The BPNN does not achieve significantly better results with usage of decomposition techniques. The error difference is statistically insignificant based on DM test.

Table 12
Training data: 5-fold cross validation results (with 95 % CI) [data with missing values imputed).

Measures	ANN		LSTM		GRU		TCN	
	Statistic	95 % CI	Statistic	95 % CI	Statistic	Statistic	95 % CI	Statistic
r	0.92	0.91 – 0.93	0.94	0.93 – 0.94	0.94	0.93 – 0.94	0.95	0.92 – 0.99
d	0.92	0.90 – 0.94	0.96	0.95 – 0.97	0.96	0.96 – 0.96	0.27	0.04 – 0.49
NSE	0.80	0.78 – 0.82	0.86	0.84 – 0.88	0.86	0.85 – 0.88	0.74	0.44 – 1.04
MAPE	56.43	42.06 – 70.80	35.85	22.27 – 49.44	26.49	20.15 – 32.83	31.91	0.13 – 0.51
RMSE	84.61	80.34 – 88.87	69.35	63.68 – 75.01	69.08	63.88 – 74.27	87.22	27.21 – 147.24
PBIAS	3.58	−5.03 – 12.19	−0.22	−11.91 – 11.47	4.47	−5.16 – −14.11	−26.75	−49.29 – −4.22
dstat	0.72	0.67 – 0.77	0.74	0.70 – 0.78	0.19	−1.47 – 1.86	0.47	0.16 – 0.78

Table 13
Test data: 5-fold cross validation results (with 95 % CI) [data with missing values imputed).

Measures	ANN		LSTM		GRU		TCN	
	Statistic	95 % CI	Statistic	95 % CI	Statistic	95 % CI	Statistic	95 % CI
r	0.86	0.85 – 0.86	0.90	0.84 – 0.96	0.90	0.84 – 0.96	0.91	0.88 – 0.94
d	0.90	0.88 – 0.91	0.93	0.90 – 0.97	0.94	0.90 – 0.97	0.27	0.02 – 0.51
NSE	0.65	0.55 – 0.75	0.79	0.69 – 0.88	0.80	0.70 – 0.90	0.67	0.43 – 0.92
MAPE	91.25	65.79 – 116.71	38.94	26.09 – 51.78	29.43	20.56 – 38.31	34.27	10.85 – 57.69
RMSE	90.07	77.99 – 102.15	78.42	52.61 – 104.22	76.33	47.01 – 105.65	93.11	67.33 – 118.89
PBIAS	−21.34	−45.89 – 3.22	−2.68	−18.30 – 12.95	3.23	−7.40 – 13.86	−26.58	−50.71 – −2.45
dstat	0.65	0.55 – 0.74	0.72	0.70 – 0.62	0.62	0.20 – 1.04	0.34	0.11 – 0.58

3. In CNN and LSTM, an interesting trend is noted. DM test reveals that, the usage of TSE decomposition degrades the accuracy of both models in comparison with the raw data and EEMD based method. The performance of the CNN and LSTM with EEMD is comparable with the results using the raw data. However, both these models (CNN and LSTM) do not produce better results than their BPNN counterparts as observed in the DM test.
4. The k-fold cross validation results show that the DL model results are stable (see the 95 % CI for the performance parameters). The DL models perform excellently. The LSTM, GRU and TCN were specifically designed for sequence data. As these (time) sequence models are also performed excellently, their use may be preferred if the data complexity increases.
5. Even though the deep learning models, LSTM, GRU, CNN and TCN are expected to outperform the BPNN, in this case for the river-flow data, this couldn't be achieved. The results of prediction with data reconstruction doesn't prove to be much different from original data, implying apparently that the data reconstruction didn't yield the desired high accuracy. Perhaps this might be because of the use of ADD activation functions in ensemble the component level predictions to the overall predictions. The work done by Xu and Niu (2022) may give some plausible reasons on why decomposition model fails in certain situations. The readers shouldn't form any decisive conclusion based on this particular result because similar procedure needs to be repeated over multiple other datasets before making generic conclusions. The present analysis thus provides a counter example to the popular expectation that ensemble empirical mode decomposition methods and deep learning algorithms can guarantee high accuracy.
6. The data characteristics analysis (DCA) also reveals that the nature of the data doesn't show appreciable changes after missing data are imputed. This is interesting and the approach considered in this study is verified. In addition, the DL models performed slightly better when missing values are imputed.

The study is not devoid of limitations. For example, in our study we have considered only the river flow data to give us predictions of future river flow patterns. However, more covariates need to be added so as to yield results that take into consideration data regarding other associated variables. Additionally, the component forecasts are added to get the ensemble forecasts. The ADD aggregation approach, though is a popular choice in literature, needs to be revisited. The use of more sophisticated aggregation function could have been explored. These remain as limitations of our study.

The results suggest that the deep learning models with data decomposition could not give better results as compared to the usage of overall data. This demands future investigations to discover and decipher the causes behind it. A future study could be conducted using different aggregation functions for ensemble predictions to check if changing the aggregation function could make ensemble-based models perform better. The present study considers the river flow measurements at one point to make water availability forecasts, however in a complex river network, the flow values are recorded at multiple measurement points and many of these measurements are correlated. For example, a high value of flow recorded at a HO station at an upstream location could cause higher values of flow at downstream locations also. The developed methodology could be modified further so that streamflow forecasting problems on such complex river systems can be handled.

In this study, we didn't consider the effect of outliers in the models' performance. We assumed that certain very high and/or very low values are expected to occur as a part of the river flow data generation process and removing them may not be advisable as the occurrences of high flows have richer information in it. The DL models should capture it. In future study, computational experiments could be done by injecting very high or very low values and predicting it effectively.

In order to help in policy making, the forecasted values need to be integrated in a decision support system in order to assess the impact of reduced flow on economic welfare of stakeholders which depend on the river water (like municipal and agricultural users) and provide appropriate policy interventions. A future analysis can be carried out to understand the association between water availability and economic welfare of agricultural and municipal users in the study area.

In recent years, the Periyar river basin has encountered heavy floods during monsoon season. Recent research (Sadhwani et al., 2022) report that this flooding is associated with Land Use Land Cover (LULC) changes in the Western Ghats region in the study area. However, a prediction model that links the Land Use Land Cover changes to streamflow is missing in the literature. This is a challenging task as LULC changes occur at a different time scale as compared to river flow. Incorporating LULC changes along with other covariates such as rainfall pattern could provide interesting new insights. We put forth these questions as potential research directions of our study, in future.

CRediT authorship contribution statement

Reetun Maiti: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Validation, Writing – review & editing. **Balagopal G. Menon:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Supervision, Validation, Writing – review & editing. **Anand Abraham:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Supervision, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix

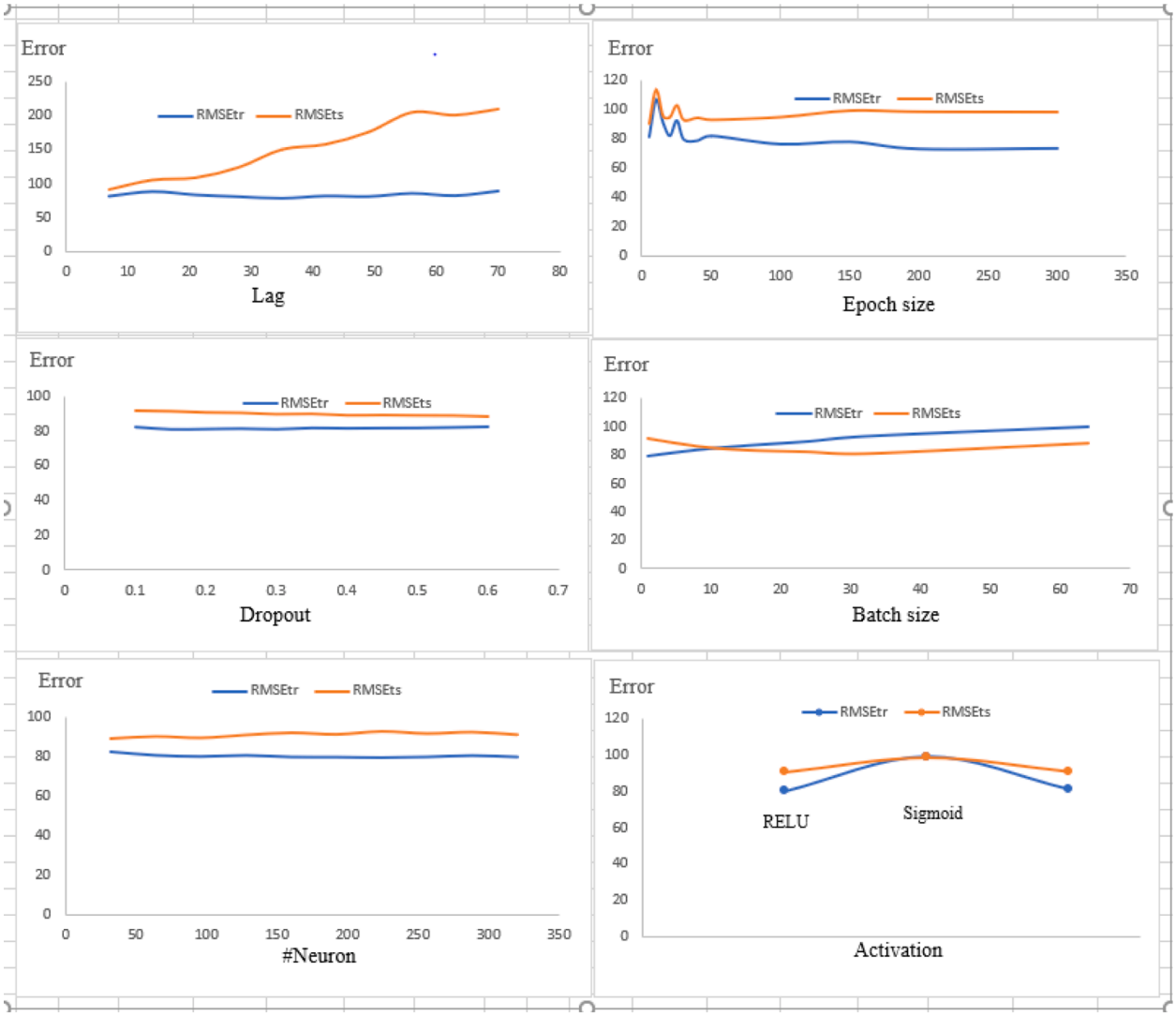


Fig. A1. LSTM parameter tuning (data with missing values deleted).

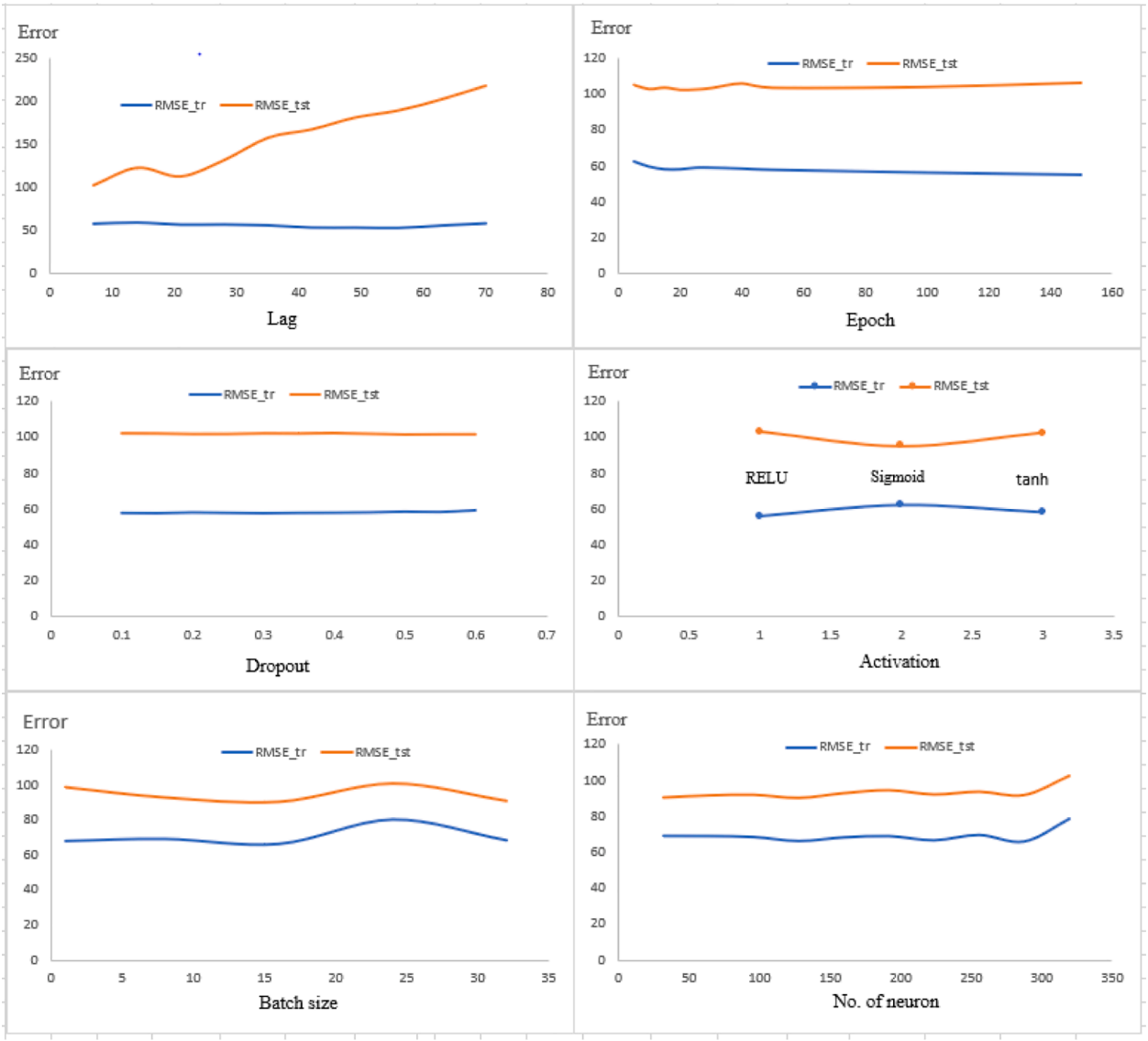


Fig. A2. LSTM parameter tuning (data with missing value imputed).

References

Ahmed, A. A. M., Deo, R. C., Ghahramani, A., Feng, Q., Raj, N., Yin, Z., & Yang, L. (2022). New double decomposition deep learning methods for river water level forecasting. *Science of The Total Environment, 831*, Article 154722.

Alizadeh, B., Bafti, A. G., Kamangir, H., Zhang, Y., Wright, D. B., & Franz, K. J. (2021). A novel attention-based LSTM cell post-processor coupled with bayesian optimization for streamflow prediction. *Journal of Hydrology, 601*, Article 126526.

Alsharef, A., Aggarwal, K., Sonia, Kumar, M., & Mishra, A. (2022). Review of ML and AutoML solutions to forecast time-series data. *Archives of Computational Methods in Engineering, 29*(7), 5297–5311.

Anusree, K., & Varghese, K. O. (2016). Streamflow prediction of Karuvannur River Basin using ANFIS, ANN and MNLR models. *Procedia Technology, 24*, 101–108.

Aslam, M., Kim, J. S., & Jung, J. (2023). Multi-step ahead wind power forecasting based on dual-attention mechanism. *Energy Reports, 9*, 239–251.

Bandt, C., & Pompe, B. (2002). Permutation entropy: A natural complexity measure for time series. *Physical review letters, 88*(17), Article 174102.

Bashir, T., Haoyong, C., Tahir, M. F., & Liqiang, Z. (2022). Short term electricity load forecasting using hybrid prophet-LSTM model optimized by BPNN. *Energy Reports, 8*, 1678–1686.

Benidis, K., Rangapuram, S. S., Flunkert, V., Wang, Y., Maddix, D., Turkmen, C., & Januschowski, T. (2022). Deep learning for time series forecasting: Tutorial and literature survey. *ACM Computing Surveys, 55*(6), 1–36.

Box, G. E. P., & Jenkins, G. M. (1970). *Time Series Analysis Forecasting and Control*. San Francisco, California, USA: Holden Day.

Brown, R. L., Durbin, J., & Evans, J. M. (1975). Techniques for testing the constancy of regression relationships over time. *Journal of the Royal Statistical Society Series B: Statistical Methodology, 37*(2), 149–163.

Carlson, R. F., MacCormick, A. J. A., & Watts, D. G. (1970). Application of linear random models to four annual streamflow series. *Water Resources Research, 6*(4), 1070–1078.

Chan, K. S., & Cryer, J. D. (2008). *Time series analysis with applications in R*. Springer Publication.

Cho, K., & Kim, Y. (2022). Improving streamflow prediction in the WRF-Hydro model with LSTM networks. *Journal of Hydrology, 605*, Article 127297.

Cho, K., Van Merriënboer, B. et al. (2014). Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation. https://arxiv.org/pdf/1406.1078v3.pdf (2014).

Chow, G. C. (1960). Tests of equality between sets of coefficients in two linear regressions. *Econometrica: Journal of the Econometric Society*, 591–605.

Coulibaly, P., & Baldwin, C. K. (2005). Nonstationary hydrological time series forecasting using nonlinear dynamic methods. *Journal of Hydrology, 307*(1–4), 164–174.

Cowpertwait, P. S., & Metcalfe, A. V. (2009). *Introductory time series with R*. Springer Science & Business Media.

Dickey, D. A., & Fuller, W. A. (1981). Likelihood ratio statistics for autoregressive time series with a unit root. *Econometrica: Journal of the Econometric Society*, 1057–1072.

Durbin, J., & Watson, G. S. (1950). Testing for serial correlation in least squares regression: I. *Biometrika, 37*(3/4), 409–428.

Fang, Y., Wang, W., Wu, P., & Zhao, Y. (2023). A sentiment-enhanced hybrid model for crude oil price forecasting. *Expert Systems with Applications, 215*, Article 119329.

Fei, S. W. (2015). Kurtosis forecasting of bearing vibration signal based on the hybrid model of empirical mode decomposition and RVM with artificial bee colony algorithm. *Expert Systems with Applications, 42*(11), 5011–5018.
Feng, Z. K., & Niu, W. J. (2021). Hybrid artificial neural network and cooperation search algorithm for nonlinear river flow time series forecasting in humid and semi-humid regions. *Knowledge-Based Systems, 211*, Article 106580.
Francis, X., & Roberto, S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics, 13*(3), 134–144.
Ghimire, S., Yaseen, Z. M., Farooque, A. A., Deo, R. C., Zhang, J., & Tao, X. (2021). Streamflow prediction using an integrated methodology based on convolutional neural network and long short-term memory networks. *Scientific Reports, 11*(1), 17497.
Gopakumar, R., Takara, K., & James, E. J. (2007). Hydrologic data exploration and river flow forecasting of a humid tropical river basin using artificial neural networks. *Water Resources Management, 21*, 1915–1940.
Gopal, M. (2022). Deep Learning Core Concepts, Methods and Applications. Pearson. PP No. 272.
Gray, H. L., Kelley, G. D., & Mc Intire, D. D. (1978). A new approach to ARMA modeling. *Communications in Statistics-Simulation and Computation, 7*(1), 1–77.
Guo, S., Wen, Y., Zhang, X., & Chen, H. (2023). Runoff prediction of lower Yellow River based on CEEMDAN–LSSVM–GM (1, 1) model. *Scientific Reports, 13*(1), 1511.
Gupta, H. V., Sorooshian, S., & Yapo, P. O. (1999). Status of automatic calibration for hydrologic models: Comparison with multilevel expert calibration. *Journal of Hydrologic Engineering, 4*(2), 135–143.
Hipel, K. W., McLeod, A. I., & Lennox, W. C. (1977). Advances in Box-Jenkins modeling: 1 Model construction. *Water Resources Research, 13*(3), 567–575.
Hunt, K. M., & Menon, A. (2020). The 2018 Kerala floods: A climate change perspective. *Climate Dynamics, 54*(3–4), 2433–2446.
Huang, N. E., Shen, Z., Long, S. R., Wu, M. C., Shih, H. H., Zheng, Q., ... & Liu, H. H. (1998). The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis. *Proceedings of the Royal Society of London. Series A: mathematical, physical and engineering sciences, 454*(1971), 903-995.
Huang, M., & Tian, Y. (2011). Hydrologic data exploration and river flow forecasting using self-organizing map and support vector regression. In *In the Fourth International Workshop on Advanced Computational Intelligence* (pp. 343–348). IEEE.
Hyndman, R. J., & Athanasopoulos, G. (2018). Forecasting: Principles and practice. *OTexts*.
James, G., Daniela, D., Trevor, H., Robert, T., & Jonathan, T. (2023). *An Introduction to Statistical Learning with Applications in Python* (p. 607p). Springer.
Jebb, A. T., Tay, L., Wang, W., & Huang, Q. (2015). Time series analysis for psychological research: Examining and forecasting change. *Frontiers in Psychology, 6*, 727.
Kai, S., Chun-qiong, L., Nan-shan, A., & Xiao-hong, Z. (2008). Using three methods to investigate time-scaling properties in air pollution indexes time series. *Nonlinear Analysis: Real World Applications, 9*(2), 693–707.
Keenan, D. M. (1985). A Tukey nonadditivity-type test for time series nonlinearity. *Biometrika, 72*(1), 39–44.
Kwiatkowski, D., Phillips, P. C., Schmidt, P., & Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root: How sure are we that economic time series have a unit root? *Journal of Econometrics, 54*(1–3), 159–178.
LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.
Lea, C., Michael D. F., Rene V., Austin R., Gregory D. H. (2016). Temporal Convolutional Networks for Action Segmentation and Detection. arXiv:1608.08242.
Li, X., Xu, W., Ren, M., Jiang, Y., & Fu, G. (2022). Hybrid CNN-LSTM models for river flow prediction. *Water Supply, 22*(5), 4902–4919.
Ljung, G. M., & Box, G. E. (1978). On a measure of lack of fit in time series models. *Biometrika, 65*(2), 297–303.
Makridakis, S., Wheelwright, S. C., & Hyndman, R. J. (1998). *Forecasting: Methods and applications*. John Wiley & Sons.
Matos, J. P., Liechti, T. C., Juízo, D., Portela, M. M., & Schleiss, A. J. (2013). Can satellite based pattern-oriented memory improve the interpolation of sparse historical rainfall records? *Journal of Hydrology, 492*, 102–116.
McLeod, A. I., Hipel, K. W., & Lennox, W. C. (1977). Advances in Box-Jenkins modeling: 2 Applications. *Water Resources Research, 13*(3), 577–586.
Modarres, R., & Eslamian, S. S. (2006). Streamflow time series modeling of Zayandehrud river. *Iranian Journal of Science & Technology, Transaction B, Engineering, 30*(B4), 565–570.
Moriasi, D. N., Arnold, J. G., Van Liew, M. W., Bingner, R. L., Harmel, R. D., & Veith, T. L. (2007). Model evaluation guidelines for systematic quantification of accuracy in watershed simulations. *Transactions of the ASABE, 50*(3), 885–900.
Packard, N. H., Crutchfield, J. P., Farmer, J. D., & Shaw, R. S. (1980). Geometry from a time series. *Physical Review Letters, 45*(9), 712.
Panahi, J., Mastouri, R., & Shabanlou, S. (2022). Insights into enhanced machine learning techniques for surface water quantity and quality prediction based on data pre-processing algorithms. *Journal of Hydroinformatics, 24*(4), 875–897.
Patel, S. S., & Ramachandran, P. (2015). A comparison of machine learning techniques for modeling river flow time series: The case of upper Cauvery river basin. *Water Resources Management, 29*, 589–602.
Phatarfod, R. M. (1989). Riverflow and reservoir storage models. *Mathematical and Computer Modelling, 12*(9), 1057–1077.
Phillips, P. C., & Perron, P. (1988). Testing for a unit root in time series regression. *Biometrika, 75*(2), 335–346.
Pulido-Calvo, I., & Portela, M. M. (2007). Application of neural approaches to one-step daily flow forecasting in Portuguese watersheds. *Journal of Hydrology, 332*(1–2), 1–15.
Rahmani, F., & Fattahi, M. H. (2022). Association between forecasting models' precision and nonlinear patterns of daily river flow time series. *Modeling Earth Systems and Environment, 8*(3), 4267–4276.
Satrio, C. B. A., Darmawan, W., Nadia, B. U., & Hanafiah, N. (2021). Time series analysis and forecasting of coronavirus disease in Indonesia using ARIMA model and PROPHET. *Procedia Computer Science, 179*, 524–532.
Sadhwani, K., Eldho, T. I., & Karmakar, S. (2022). Flooding Problems in Periyar River Basin, Kerala—The Effects of Land Use Land Cover Changes. In *5th World Congress on Disaster Management* (Volume II, pp. 16–24). Routledge.
Salas, J. D., & Obeysekera, J. T. B. (1982). ARMA model identification of hydrologic time series. *Water Resources Research, 18*(4), 1011–1021.
Samui, P., Yesilyurt, S. N., Dalkilic, H. Y., Yaseen, Z. M., Roy, S. S., & Kumar, S. (2023). Comparison of different optimized machine learning algorithms for daily river flow forecasting. *Earth Science Informatics, 16*(1), 533–548.
Singh, N., & Chinnasamy, P. (2021). Non-stationary flood frequency analysis and attribution of streamflow series: A case study of periyar river, india. *Hydrological Sciences Journal, 66*(13), 1866–1881.
Sun, X., Zhang, H., Wang, J., Shi, C., Hua, D., & Li, J. (2022). Ensemble streamflow forecasting based on variational mode decomposition and long short term memory. *Scientific Reports, 12*(1), 518.
Tang, L., Yu, L., Liu, F., & Xu, W. (2013). An integrated data characteristic testing scheme for complex time series data exploration. *International Journal of Information Technology & Decision Making, 12*(03), 491–521.
Theiler, J., Eubank, S., Longtin, A., Galdrikian, B., & Farmer, J. D. (1992). Testing for nonlinearity in time series: The method of surrogate data. *Physica D: Nonlinear Phenomena, 58*(1–4), 77–94.
Tsay, R. S. (1986). Nonlinearity tests for time series. *Biometrika, 73*(2), 461–466.
Unsworth, C. P., Cowper, M. R., McLaughlin, S., & Mulgrew, B. (2001). A new method to detect nonlinearity in a time-series: Synthesizing surrogate data using a Kolmogorov-Smirnoff tested, hidden Markov model. *Physica D: Nonlinear Phenomena, 155*(1–2), 51–68.
Valipour, M., Banihabib, M. E., & Behbahani, S. M. R. (2012). Parameters estimate of autoregressive moving average and autoregressive integrated moving average models and compare their ability for inflow forecasting. *Journal of Mathematics and Statistics, 8*(3), 330–338.
Vijayan, A., Praveen, S. S., & Babu, V. S. (2021). Simulation Studies of Flood Along the Periyar River Basin, Central Kerala, India. *Journal of Advanced Research in Geo Sciences & Remote Sensing, 8*(2), 1–7.
Wang, J., & Duggasani, A. (2020). Forecasting hotel reservations with long short-term memory-based recurrent neural networks. *International Journal of Data Science and Analytics, 9*(1), 77–94.
Wang, J., Zhang, L., Liu, Z., & Niu, X. (2022). A novel decomposition-ensemble forecasting system for dynamic dispatching of smart grid with sub-model selection and intelligent optimization. *Expert Systems with Applications, 201*, Article 117201.
Wang, S., Yu, L., Tang, L., & Wang, S. (2011). A novel seasonal decomposition based least squares support vector regression ensemble learning approach for hydropower consumption forecasting in China. *Energy, 36*(11), 6542–6554.
West, D., & Dellana, S. (2011). An empirical analysis of neural network memory structures for basin water quality forecasting. *International Journal of Forecasting, 27*(3), 777–803.
Willmott, C. J. (1981). On the validation of models. *Physical geography, 2*(2), 184–194.
Wu, J., Wang, Z., Hu, Y., Tao, S., & Dong, J. (2023). Runoff forecasting using convolutional neural networks and optimized bi-directional long short-term memory. *Water Resources Management, 37*(2), 937–953.
Xu, K., & Niu, H. (2022). Do EEMD based decomposition-ensemble models indeed improve prediction for crude oil futures prices? *Technological Forecasting and Social Change, 184*, Article 121967.
Yang, Y., & Wang, J. (2021). Forecasting wavelet neural hybrid network with financial ensemble empirical mode decomposition and MCID evaluation. *Expert Systems with Applications, 166*, Article 114097.
Yenidoğan, I., Çayir, A., Kozan, O., Dağ, T., & Arslan, Ç. (2018, September). Bitcoin forecasting using ARIMA and PROPHET. In *2018 3rd international conference on computer science and engineering (UBMK)* (pp. 621-624). IEEE.
Yu, L., Wang, Z., & Tang, L. (2015). A decomposition–ensemble model with data-characteristic-driven reconstruction for crude oil price forecasting. *Applied Energy, 156*, 251–267.
Yu, P. S., Chen, S. T., & Chang, I. F. (2006). Support vector regression for real-time flood stage forecasting. *Journal of Hydrology, 328*(3–4), 704–716.
Zeileis, A. (2001). *p values and alternative boundaries for CUSUM tests* (No. 2001, 02). Technical Report.
Zeileis, A. (2005). A unified approach to structural change tests based on ML scores, *F* statistics, and OLS residuals. *Econometric Reviews, 24*(4), 445–466.
Zhang, G. P. (2001). An investigation of neural networks for linear time-series forecasting. *Computers & Operations Research, 28*(12), 1183–1202.
Zuo, G., Luo, J., Wang, N., Lian, Y., & He, X. (2020). Decomposition ensemble model based on variational mode decomposition and long short-term memory for streamflow forecasting. *Journal of Hydrology, 585*, Article 124776.
Zou, Y., Yu, L., & He, K. (2023). Forecasting crude oil risk: A multiscale bidirectional generative adversarial network based approach. *Expert Systems with Applications, 212*, Article 118743.